



**Detección de Alojamientos
Turísticos con Precio Real Inferior
al Valor Predicho según
Preferencias del Usuario**

Mario Jiménez León



Detección de Alojamientos Turísticos con Precio Real Inferior al Valor Predicho según Preferencias del Usuario

Mario Jiménez León

Memoria presentada como parte de los requisitos para la obtención del título de Grado en Estadística por la Universidad de Sevilla.

Tutorizada por

David Orellana Martín

Índice general

Resumen	1
Abstract	2
Índice de figuras	2
Índice de cuadros	3
 I Introducción	 5
1. Introducción	6
1.1. Motivación	6
1.2. Objetivos	7
1.3. Alcance y limitaciones	8
1.4. Estructura de la memoria	9
 2. Estado del Arte	 10
2.1. El mercado de alojamiento turístico <i>online</i>	10
2.2. Extracción de datos de plataformas web	11
2.3. Predicción de precios hoteleros mediante aprendizaje automático .	11
2.4. Detección de anomalías de precio	12
2.5. Herramientas de despliegue de aplicaciones analíticas	12
2.6. Posicionamiento del presente trabajo	13

II	Estudio Teórico	14
3.	Fundamentos Estadísticos	15
3.1.	Variables aleatorias y distribuciones	15
3.1.1.	Distribución normal	16
3.1.2.	Distribución log-normal	16
3.2.	Estadísticos descriptivos	16
3.3.	Detección de valores atípicos: el método de Tukey	17
3.4.	Correlación lineal de Pearson	18
3.5.	Normalización y estandarización	18
3.5.1.	Normalización mín-máx	19
3.5.2.	Estandarización <i>z-score</i>	19
3.6.	Codificación de variables categóricas	19
3.6.1.	Codificación ordinal	19
3.6.2.	Codificación <i>one-hot</i>	20
4.	Modelos de Regresión	21
4.1.	El problema de regresión	21
4.2.	Regresión lineal múltiple	21
4.2.1.	Estimación por mínimos cuadrados ordinarios	22
4.2.2.	Supuestos e hipótesis	22
4.3.	Árbol de decisión para regresión	23
4.3.1.	Algoritmo CART	23
4.3.2.	Sobreajuste y poda	23
4.4.	Bosque Aleatorio	23
4.4.1.	Construcción del <i>ensemble</i>	24
4.4.2.	Importancia de las variables	24
4.4.3.	Hiperparámetros	25
4.5.	<i>Gradient Boosting</i> y XGBoost	25

4.5.1. Formulación general	25
4.5.2. XGBoost	25
4.6. Métricas de evaluación para regresión	26
5. Validación de Modelos y Categorización de Chollos	28
5.1. Partición del conjunto de datos	28
5.2. Validación cruzada y búsqueda de hiperparámetros	29
5.2.1. Validación cruzada de k pliegues	29
5.2.2. Búsqueda en rejilla	30
5.2.3. Estandarización y filtración de información	30
5.3. Categorización determinista de chollos	31
5.3.1. Justificación: categorización determinista frente a clasifica- dor ML	31
III Desarrollo Práctico	33
6. Sistema de Extracción y Preprocesamiento de Datos	34
6.1. Fuente y recopilación de los datos	34
6.1.1. Geocodificación de los centros urbanos	35
6.2. Preprocesamiento de los datos	35
6.2.1. Tratamiento de valores nulos	35
6.2.2. Detección y eliminación de valores atípicos	36
6.2.3. Extracción de servicios	39
6.3. Ingeniería de características	39
6.3.1. Variables temporales	39
6.3.2. Codificación de variables categóricas	40
6.4. Estadísticas descriptivas del conjunto de datos	40
6.5. Análisis de correlaciones	42

7. Modelización, Resultados y Aplicación Web	45
7.1. Diseño experimental	45
7.1.1. Partición y tamaño de los conjuntos	45
7.1.2. Vector de características	45
7.1.3. Rejilla de hiperparámetros	46
7.2. Resultados de los modelos de regresión	46
7.2.1. Variable raíz del árbol de decisión	48
7.2.2. Importancia de las variables en el bosque aleatorio	49
7.3. Distribución de categorías de chollo	49
7.4. Aplicación web	50
7.4.1. Arquitectura de la aplicación	51
7.4.2. Flujo de trabajo de la página de búsqueda	51
7.4.3. Optimización de memoria: poda del modelo para el despliegue	52
7.4.4. Despliegue	54
7.4.5. Limitaciones de la extracción en tiempo real	54
7.5. Conclusiones	54
7.6. Trabajo futuro	55
Bibliografía	57

Resumen

Este trabajo presenta un sistema automatizado para la detección de alojamientos turísticos cuyo precio real de oferta es inferior al valor estimado por un modelo predictivo, es decir, alojamientos que constituyen una oportunidad genuina respecto a sus características objetivas. La búsqueda se realiza según las preferencias del usuario (destino, fechas, tipo de alojamiento y servicios deseados) a través de una aplicación web interactiva, que devuelve los alojamientos categorizados según su grado de oportunidad. Los datos de alojamientos y sus atributos se recopilaban de una plataforma de reservas, cubriendo las ocho provincias de la comunidad autónoma de Andalucía. Tras una fase exhaustiva de preprocesamiento e ingeniería de características, se entrenaron cuatro modelos de regresión para predecir el precio justo de mercado y categorizar cada alojamiento según la magnitud de la diferencia entre su precio real y el valor predicho. El mejor modelo fue un bosque aleatorio (*Random Forest*) con coeficiente de determinación $R^2 = 0,87$ y un error absoluto medio de aproximadamente 19,43 € por noche.

Abstract

This work presents an automated system for detecting tourist accommodations whose actual listed price is lower than the value estimated by a predictive model — that is, accommodations that constitute a genuine opportunity relative to their objective characteristics. The search is carried out according to the user's preferences (destination, dates, accommodation type, and desired services) through an interactive web application, which returns the accommodations categorised by their degree of opportunity. Accommodation listings and their attributes were collected from a booking platform, covering the eight provinces of the autonomous community of Andalusia. After an exhaustive preprocessing and feature engineering phase, four regression models were trained to predict the fair market price and categorise each accommodation according to the magnitude of the difference between its actual price and the predicted value. The best-performing model was a Random Forest regressor with a coefficient of determination $R^2 = 0.87$ and a mean absolute error of approximately 19.43 € per night.

Índice de figuras

6.1. Diagrama de caja del precio por noche antes de la eliminación de valores atípicos.	37
6.2. Diagrama de caja del tamaño de la habitación por tipo de alojamiento antes de la eliminación de valores atípicos.	37
6.3. Diagrama de caja de la distancia al centro urbano antes (izquierda) y después (derecha) de eliminar el error de geocodificación de Guadalcanal.	38
6.4. Muestra de los datos en bruto de Málaga	41
6.5. Muestra del conjunto procesado total (incluye las ocho provincias)	41
6.6. Matriz de correlaciones de Pearson sobre el <i>dataset</i> codificado (26 variables).	44
7.1. Diagrama de dispersión de <code>tamaño_habitacion</code> (m ²) frente a <code>precio</code> (€) sobre el conjunto de entrenamiento.	47
7.2. Árbol de decisión de regresión: primeros 4 niveles.	48
7.3. Distribución de las cinco categorías de chollo en el conjunto de prueba (390.462 registros).	50

Índice de cuadros

5.1. Categorías de chollo según el cociente r entre precio real y precio predicho.	31
6.1. Estrategia de imputación de valores nulos por variable.	36
6.2. Registros eliminados en cada etapa del filtrado de valores atípicos.	39
6.3. Estadísticos descriptivos de las variables continuas del conjunto de datos.	42
7.1. Tamaño de las particiones del conjunto de datos.	45
7.2. Rejilla de hiperparámetros para cada modelo de regresión.	46
7.3. Resultados de los modelos de regresión sobre el conjunto de validación.	46
7.4. Distribución de categorías de chollo en el conjunto de prueba (390.462 registros).	49
7.5. Rango de ahorro exigido por categoría según el nivel de exigencia elegido por el usuario.	52
7.6. Comparación del bosque aleatorio completo y de la versión podada (20 árboles) sobre el conjunto de prueba.	53

Parte I

Introducción

1 | Introducción

El turismo constituye uno de los pilares fundamentales de la economía española. Según el Instituto Nacional de Estadística, España recibió en 2023 más de 85 millones de turistas internacionales, consolidándose como uno de los destinos más visitados del mundo. Dentro del panorama nacional, Andalucía ocupa un lugar destacado tanto por su patrimonio cultural y natural como por la diversidad de su oferta de alojamiento, que abarca desde hoteles de gran lujo hasta estudios y apartamentos de bajo coste.

La irrupción de las plataformas de reserva *online*, denominadas habitualmente *Online Travel Agencies* (OTA), ha transformado radicalmente la manera en que viajeros y propietarios interactúan en el mercado de alojamiento. Estas plataformas agregan miles de listados de distintas categorías y localizaciones, y aplican estrategias de precio dinámico que hacen fluctuar las tarifas en función de la demanda, la estacionalidad, los días que restan hasta la estancia o incluso el historial de búsqueda del usuario. Esta complejidad dificulta al viajero medio identificar si el precio que observa en pantalla es realmente competitivo o, por el contrario, está artificialmente inflado.

En este contexto surge la noción coloquial de «chollo»: un alojamiento cuyo precio de oferta se sitúa notablemente por debajo de lo que cabría esperar atendiendo a sus características objetivas (ubicación, estrellas, valoraciones, servicios incluidos, etc.). Detectar estos chollos de forma sistemática y en tiempo real es una tarea que, por su volumen y complejidad, escapa a las posibilidades de cualquier búsqueda manual, pero resulta perfectamente abordable mediante técnicas de aprendizaje automático entrenadas sobre datos extraídos de las propias plataformas.

1.1 Motivación

La motivación principal de este trabajo nace de la confluencia de dos realidades. Por un lado, la enorme cantidad de datos de precios disponibles públicamente en las OTA, que constituyen un recurso valioso para modelar el comportamiento del mercado de alojamiento turístico. Por otro, la madurez que han alcanzado las técnicas de aprendizaje automático para tareas de predicción sobre datos tabulares de alta dimensionalidad, con algoritmos como los bosques aleatorios o el *gradient*

boosting capaces de capturar relaciones no lineales complejas entre variables.

Más allá del interés académico, existe una motivación práctica clara: proporcionar al viajero una herramienta que, introduciendo sus preferencias de destino, fechas, tipo de alojamiento y servicios, devuelva en tiempo real una lista ordenada de opciones con una etiqueta que cuantifique el grado en que cada precio se desvía del valor justo estimado.

Desde el punto de vista matemático, el problema presenta una estructura bien definida que lo hace especialmente atractivo para un Trabajo de Fin de Grado en Estadística: un problema de regresión para estimar el precio justo de mercado, junto a su posterior categorización en cada alojamiento según la magnitud de esa desviación. Este problema requiere un tratamiento riguroso de los datos, un conocimiento sólido de los modelos estadísticos y de aprendizaje automático empleados, y una evaluación cuidadosa de los resultados.

1.2 Objetivos

El objetivo general del trabajo es el diseño, implementación y evaluación de un sistema integral capaz de detectar, en Andalucía, alojamientos turísticos con precio real inferior al valor predicho, a partir de datos obtenidos de una plataforma de reservas de amplia difusión.

Este objetivo general se desglosa en los siguientes objetivos específicos:

1. Construir un *pipeline* de extracción de datos que obtenga listados de alojamientos y sus características detalladas (precio por noche, tipo de alojamiento, número de estrellas, valoración media, número de reseñas, servicios del alojamiento, servicios de la habitación y coordenadas geográficas) para las ocho provincias andaluzas.
2. Diseñar y aplicar un protocolo de preprocesamiento y limpieza de datos que incluya el tratamiento de valores nulos, la detección y eliminación de valores atípicos mediante métodos estadísticos robustos, la extracción de características binarias a partir de campos de texto libre, y el cálculo de variables derivadas como la distancia al centro urbano o indicadores temporales.
3. Entrenar y comparar cuatro modelos de regresión (regresión lineal múltiple, árbol de decisión, bosque aleatorio y *gradient boosting*) para predecir el precio justo de cada alojamiento, y seleccionar el de mejor rendimiento mediante validación cruzada y evaluación en un conjunto de prueba reservado.
4. Diseñar una regla determinista de categorización que, a partir del cociente entre el precio real y el precio predicho por el modelo de regresión, asigne a cada alojamiento una etiqueta de categoría (desde «inflado» hasta «hiper-chollo»), evitando así la redundancia de entrenar un clasificador de

aprendizaje automático sobre información ya contenida en la predicción de regresión.

5. Desarrollar una aplicación web interactiva que integre el sistema en un flujo de trabajo de extremo a extremo: el usuario especifica sus preferencias, la aplicación extrae datos en tiempo real, aplica el modelo y presenta los resultados ordenados por grado de oportunidad.

1.3 Alcance y limitaciones

El sistema desarrollado se centra exclusivamente en alojamientos localizados en las ocho provincias de la comunidad autónoma de Andalucía: Almería, Cádiz, Córdoba, Granada, Huelva, Jaén, Málaga y Sevilla. Esta delimitación geográfica responde tanto a la coherencia del proyecto como a restricciones prácticas de tiempo y capacidad computacional en la fase de extracción de datos.

Los datos empleados para el entrenamiento de los modelos fueron extraídos en un intervalo temporal concreto; dado que los precios en las plataformas OTA son altamente dinámicos, los modelos capturan el estado del mercado en ese momento y pueden no reflejar con total exactitud fluctuaciones estacionales extremas fuera de la ventana de extracción.

El objetivo de predicción del sistema es el precio de un alojamiento concreto en una fecha concreta, dado que los precios en plataformas OTA son dinámicos y varían según la fecha de la estancia. Por ello, la partición en entrenamiento, validación y prueba (Sección 5.1) se realiza a nivel de fila (alojamiento-fecha), sin agrupar por alojamiento ni respetar el orden temporal del calendario de precios; además, el identificador del alojamiento no forma parte de las variables del modelo, por lo que este no puede reconocer explícitamente una observación como perteneciente a un alojamiento visto en entrenamiento. No obstante, dado que las filas de un mismo alojamiento comparten prácticamente todos los atributos estáticos (estrellas, servicios, localización, tamaño) y solo difieren en las variables temporales y en el precio, entrenamiento y prueba no son completamente independientes cuando un mismo alojamiento aparece en ambos; esto puede hacer que las métricas de rendimiento reportadas en el Capítulo 7 resulten ligeramente optimistas respecto al comportamiento del modelo frente a alojamientos completamente inéditos.

Durante el preprocesamiento se detectó que un 0,026 % de los registros (674 de 2.603.076), correspondientes a tres de las ocho provincias, presenta un valor de -1 en la variable `días_restantes` hasta la estancia. La causa es que el proceso de extracción de datos original no registró la fecha de extracción de cada ficha; para suplir esta carencia, la variable `fecha_extraccion` se añadió posteriormente en el preprocesamiento, asignando manualmente una fecha fija por provincia en lugar de la fecha real de extracción, y para ese pequeño subconjunto de fichas la fecha de disponibilidad cae un día antes de la fecha asignada a su provincia. Dado su volumen marginal, no se considera que comprometa la validez de los resultados,

pero se documenta aquí por transparencia.

Por prudencia, en esta memoria no se identifica explícitamente la plataforma de la que proceden los datos. Aún así, cabe aclarar que el proyecto se plantea únicamente con fines académicos y de investigación, sin ningún propósito comercial.

1.4 Estructura de la memoria

La memoria se organiza en tres partes principales, además de esta introducción y las conclusiones.

La **Parte I** (Introducción) incluye, además de este capítulo, el Capítulo 2, que revisa el estado del arte en extracción de datos de plataformas de alojamiento, en predicción de precios hoteleros mediante aprendizaje automático, y en detección de anomalías de precio.

La **Parte II** (Estudio Teórico) comprende los tres capítulos siguientes. El Capítulo 3 presenta los fundamentos estadísticos sobre los que se sustenta el trabajo: distribuciones de probabilidad, estadísticos descriptivos, detección de valores atípicos y medidas de correlación lineal. El Capítulo 4 detalla los cuatro modelos de regresión utilizados, desde la regresión lineal clásica hasta el *gradient boosting*, describiendo su formulación matemática y las métricas de evaluación empleadas. El Capítulo 5 aborda los métodos de validación (partición de datos, validación cruzada y búsqueda de hiperparámetros) y la categorización determinista de chollos a partir del cociente entre precio real y precio predicho.

La **Parte III** (Desarrollo Práctico) se divide en dos capítulos. El Capítulo 6 describe la fuente y recopilación de los datos, el proceso de preprocesamiento e ingeniería de características, y las estadísticas descriptivas del conjunto de datos resultante. El Capítulo 7 presenta los experimentos de modelización, los resultados obtenidos y la aplicación web desarrollada para el despliegue del sistema.

Finalmente, la memoria recoge las referencias bibliográficas utilizadas a lo largo del trabajo.

El código completo de este trabajo, incluyendo la extracción de datos, el preprocesamiento, la modelización y la aplicación web, está disponible públicamente en el repositorio <https://github.com/Marjimleo3/Trabajo-Fin-de-Grado> [16].

2 | Estado del Arte

En este capítulo se sitúa el presente trabajo en su contexto científico y tecnológico, organizado en tres bloques. El primero, «El mercado de alojamiento turístico *online*», presenta el fenómeno del precio dinámico que motiva la necesidad de este trabajo. El segundo revisa los trabajos relacionados en tres frentes: «Extracción de datos de plataformas web», «Predicción de precios hoteleros mediante aprendizaje automático» y «Detección de anomalías de precio». El tercero, que comprende «Herramientas de despliegue de aplicaciones analíticas» y «Posicionamiento del presente trabajo», describe las tecnologías empleadas para el despliegue del sistema y sitúa las aportaciones de este trabajo respecto a las alternativas identificadas en la literatura.

2.1 El mercado de alojamiento turístico *online*

El mercado de alojamiento turístico ha experimentado una profunda transformación desde la aparición de las plataformas de reserva *online* a finales de los años noventa. Portales como Booking.com, Expedia o Airbnb han pasado a intermediar una parte muy relevante de las reservas de alojamiento a nivel mundial, proporcionando a los viajeros acceso simultáneo a millones de listados y a los propietarios canales de distribución de alcance global [6].

Una de las características definitorias de este mercado es el precio dinámico (*dynamic pricing*): la tarifa de un mismo alojamiento para una misma fecha puede variar varias veces al día en función de la demanda, la disponibilidad remanente, la anticipación de la reserva y otros factores como la localización del usuario o sus búsquedas previas. Este fenómeno, bien conocido desde el sector aéreo bajo el nombre de *yield management*, fue adoptado progresivamente por la industria hotelera a lo largo de los años 2000 y se ha extendido de forma masiva a través de las OTA [31].

El precio dinámico plantea un reto de información asimétrica: el proveedor de alojamiento dispone de información privilegiada sobre su propia estrategia de precios y sobre la ocupación esperada, mientras que el viajero solo observa el precio publicado en un instante concreto sin posibilidad de saber si es alto o bajo respecto al valor esperable del producto. Esta asimetría justifica el interés de

herramientas que, mediante el análisis de grandes volúmenes de datos históricos, permitan estimar el precio «justo» de un alojamiento en función de sus atributos observables.

2.2 Extracción de datos de plataformas web

El *web scraping* es una técnica que permite recuperar información estructurada a partir de páginas web mediante programas que replican la navegación de un usuario en un navegador [21]. Las dos herramientas para el control programático de navegadores más extendidas en la actualidad son Selenium [28] y Playwright [19].

En el contexto de las OTA, varios trabajos han empleado técnicas de *web scraping* para construir conjuntos de datos de precios hoteleros. [27] construyeron un gran conjunto de datos de precios de Booking.com para hoteles de tres, cuatro y cinco estrellas en Madrid y analizaron los determinantes del precio por categoría de establecimiento. [33] construyeron un corpus de reseñas de TripAdvisor para analizar la correspondencia entre el sentimiento expresado en el texto y la puntuación numérica otorgada por el usuario. Más recientemente, [18] emplearon datos del mercado de alquiler entre particulares (*peer-to-peer*) en Barcelona para estudiar el impacto inicial de la pandemia de COVID-19 en los precios del alojamiento de corta estancia, encontrando caídas de precio del 11,3 % en viviendas completas y del 4,7 % en habitaciones individuales.

2.3 Predicción de precios hoteleros mediante aprendizaje automático

La predicción de precios de alojamientos turísticos es un problema de regresión supervisada en el que las variables explicativas recogen los atributos del alojamiento (tipo, categoría, localización, servicios disponibles, valoración de los usuarios) y las condiciones de la reserva (fechas, anticipación), mientras que la variable respuesta es el precio por noche.

Los primeros enfoques en la literatura aplicaron modelos econométricos clásicos. [1] emplearon modelos de precios hedónicos para analizar la formación de precios en hoteles italianos, identificando que variables como la categoría por estrellas, la localización y el número de reseñas *online* son determinantes primarios. [10] estudiaron las estrategias de *yield management* en hoteles estadounidenses, comparando sus tarifas con las de la competencia directa.

Con la irrupción del aprendizaje automático, los investigadores comenzaron a aplicar modelos más flexibles capaces de capturar relaciones no lineales. [14] compararon seis algoritmos (bosque aleatorio, XGBoost, regresión lineal múltiple,

SVR, MLP y KNN) para modelar el precio de hoteles económicos en Sanya (China) desde una perspectiva espacial. [26] aplicaron *gradient boosting* a datos de Airbnb en Nueva York, incorporando variables de texto extraídas mediante análisis de sentimiento de las reseñas de los huéspedes junto a los atributos estructurados del alojamiento.

En el contexto español, [29] aplicaron un modelo de precios hedónicos con regresión ponderada geográficamente (GWR) a 57 hoteles de Málaga, combinando variables del establecimiento (categoría, tamaño, localización) con las valoraciones de sus servicios, y encontraron una heterogeneidad espacial relevante en la relación entre calidad percibida y precio dentro de una misma ciudad, precisamente una de las ocho provincias analizadas en este trabajo.

2.4 Detección de anomalías de precio

El problema de detectar alojamientos con precios anómalos puede encuadrarse dentro del campo más amplio de la detección de anomalías (*anomaly detection*). En términos generales, una anomalía de precio es una observación cuyo valor se desvía de forma significativa de lo esperado dado el conjunto de atributos observados. Existen dos enfoques principales para abordar este problema.

El primer enfoque es no supervisado: se aprende una distribución sobre los precios a partir de los datos y se etiquetan como anómalas las observaciones en los extremos de esa distribución. Los métodos más comunes incluyen los bosques de aislamiento (*Isolation Forest*) [17], el factor de valor atípico local (*Local Outlier Factor*) [5] y los autocodificadores (*autoencoders*) basados en redes neuronales [13, 8].

El segundo enfoque (el adoptado en este trabajo) es supervisado: se entrena un modelo de regresión para predecir el precio esperado y se compara el precio observado con la predicción. La desviación relativa entre ambos sirve de indicador del grado de anomalía, y una regla determinista aplicada sobre esa desviación asigna a cada alojamiento una etiqueta de categoría (Sección 5.3), sin necesidad de entrenar un clasificador adicional. Este enfoque tiene la ventaja de que la noción de anomalía queda anclada en los atributos del alojamiento, lo que hace que el enfoque sea más interpretable para el usuario final [2].

2.5 Herramientas de despliegue de aplicaciones analíticas

El componente de despliegue del sistema, la aplicación web, se ha construido con Streamlit [30], un *framework* de Python diseñado específicamente para crear

interfaces web de análisis de datos y aprendizaje automático de forma rápida y con escaso código de interfaz. A diferencia de *frameworks* web de propósito general como Flask o Django, Streamlit gestiona automáticamente el ciclo de vida de la aplicación y ofrece *widgets* interactivos predefinidos. Su popularidad en la comunidad científica y de ingeniería de datos ha crecido de forma notable desde su lanzamiento en 2019, con casos de uso que abarcan desde exploradores de datos hasta asistentes con modelos de lenguaje.

2.6 Posicionamiento del presente trabajo

En comparación con los trabajos revisados, el presente proyecto presenta varias características diferenciadoras: combina en un único sistema integrado la extracción de datos en tiempo real, el preprocesamiento, la predicción y la presentación al usuario, mientras que la mayoría de trabajos en la literatura se centran en alguna de estas fases de forma aislada. Además, el sistema opera sobre datos del mercado andaluz: aunque [29] aplicó un modelo de regresión ponderada geográficamente a una muestra estática de hoteles malagueños, hasta donde se ha podido comprobar en la literatura revisada no se han encontrado estudios que apliquen modelos de aprendizaje automático de conjunto (*ensemble*) sobre un horizonte temporal de precios y a la granularidad de listado individual en el conjunto de las ocho provincias andaluzas. Por último, la arquitectura adoptada permite al usuario personalizar la búsqueda según sus preferencias, lo que añade un componente de utilidad práctica que va más allá del análisis académico.

Parte II

Estudio Teórico

3 | Fundamentos Estadísticos

Antes de abordar el preprocesamiento y el modelado, este capítulo recoge los fundamentos matemáticos y estadísticos sobre los que se apoya el tratamiento de datos realizado en este trabajo. Se presentan los conceptos de variable aleatoria y distribución de probabilidad, los estadísticos descriptivos empleados en el análisis exploratorio, el método de Tukey para la detección de valores atípicos, y las medidas de correlación lineal usadas para estudiar la relación entre variables. Finalmente se describen las técnicas de normalización y estandarización de variables numéricas, y de codificación de variables categóricas, empleadas en el preprocesamiento.

3.1 Variables aleatorias y distribuciones

Definición 3.1. Sea $(\Omega, \mathcal{F}, P)$ un espacio de probabilidad. Una variable aleatoria es una función $X: \Omega \rightarrow \mathbb{R}$ medible respecto a $\mathcal{F}$, es decir, tal que $\{X \leq x\} \in \mathcal{F}$ para todo $x \in \mathbb{R}$.

La distribución de una variable aleatoria queda caracterizada por su función de distribución acumulada $F_X(x) = P(X \leq x)$. Cuando X es absolutamente continua, existe una función de densidad f_X tal que

$$F_X(x) = \int_{-\infty}^x f_X(t) dt.$$

En el contexto de este trabajo, las variables de interés son principalmente continuas (precios, distancias, valoraciones) o discretas (número de reseñas, indicadores binarios de servicios). Dado que los precios de alojamiento no pueden tomar valores negativos y presentan una fuerte asimetría positiva, sus distribuciones se asemejan más a distribuciones log-normales que a distribuciones normales, lo que tiene implicaciones directas en el tratamiento de valores atípicos y en la evaluación de modelos de regresión; de hecho, es uno de los motivos del bajo rendimiento de la regresión lineal frente a los modelos de árboles en este trabajo.

3.1.1 Distribución normal

La distribución normal o gaussiana, denotada $\mathcal{N}(\mu, \sigma^2)$, es una de las más utilizadas en estadística inferencial por sus propiedades analíticas y por el Teorema Central del Límite. Su función de densidad es

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right), \quad x \in \mathbb{R},$$

donde $\mu \in \mathbb{R}$ es la media y $\sigma^2 > 0$ la varianza. Aunque los precios de alojamiento no siguen esta distribución, la normalidad es relevante para la inferencia clásica (contrastes de hipótesis e intervalos de confianza) sobre los residuos de los modelos de regresión lineal, pero no para la calidad de sus predicciones.

3.1.2 Distribución log-normal

Una variable aleatoria $X > 0$ sigue una distribución log-normal si $\ln X \sim \mathcal{N}(\mu, \sigma^2)$. Su función de densidad es

$$f(x) = \frac{1}{x\sigma\sqrt{2\pi}} \exp\left(-\frac{(\ln x - \mu)^2}{2\sigma^2}\right), \quad x > 0.$$

Las distribuciones log-normales son habituales en variables económicas como precios o ingresos, donde la variable no puede ser negativa y presenta una distribución sesgada a la derecha (asimetría positiva). En este trabajo, sin embargo, se optó por no aplicar una transformación logarítmica al precio, tal como se justifica en la Sección 3.3.

3.2 Estadísticos descriptivos

El análisis exploratorio de los datos requiere un conjunto de estadísticos que resuman la distribución de cada variable [7, 32].

Definición 3.2. Sea $\{x_1, \dots, x_n\}$ una muestra de tamaño n . La media muestral es

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i.$$

La varianza muestral es

$$s^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2,$$

y la desviación típica muestral es $s = \sqrt{s^2}$.

La media es sensible a valores extremos, lo que la hace poco representativa cuando la distribución es asimétrica o contiene *outliers*. En esos casos es preferible emplear la mediana, que es el valor que ocupa la posición central en la muestra ordenada.

| Definición 3.3. *Los cuartiles Q_1 , Q_2 y Q_3 son los percentiles 25, 50 y 75 de la muestra, respectivamente. El rango intercuartílico (IQR, de sus siglas en inglés) se define como*

$$\text{IQR} = Q_3 - Q_1.$$

El rango intercuartílico mide la dispersión de la mitad central de los datos y es resistente a la influencia de los valores extremos, lo que lo convierte en un estadístico de escala robusto.

3.3 Detección de valores atípicos: el método de Tukey

Los valores atípicos (*outliers*) son observaciones que se alejan significativamente del patrón general de los datos. Su presencia puede deberse a errores de medición o registro, a casos genuinamente excepcionales, o a ambas causas a la vez. En cualquier caso, su tratamiento adecuado es crucial para el correcto entrenamiento de los modelos predictivos, pues muchos de ellos son sensibles a valores extremos.

El método de Tukey [32] es uno de los más extendidos para la detección de *outliers* en datos univariantes. Se basa en el rango intercuartílico y no asume ninguna distribución paramétrica concreta para los datos.

| Definición 3.4 (Criterio de Tukey). *Sea $\{x_1, \dots, x_n\}$ una muestra con cuartiles Q_1 y Q_3 y rango intercuartílico $\text{IQR} = Q_3 - Q_1$. Se definen los límites de Tukey:*

$$L_{\text{inf}} = Q_1 - 1,5 \cdot \text{IQR}, \quad L_{\text{sup}} = Q_3 + 1,5 \cdot \text{IQR}.$$

Una observación x_i se considera valor atípico si $x_i < L_{\text{inf}}$ o $x_i > L_{\text{sup}}$.

El factor 1,5 elegido por Tukey no es arbitrario: bajo una distribución normal, los límites así definidos dejan fuera aproximadamente el 0,7 % de las observaciones, de modo que valores más extremos tienen una probabilidad muy baja de aparecer en muestras normalmente distribuidas. En distribuciones asimétricas, como las de precios, el método puede resultar excesivamente agresivo en la cola larga; en esos casos es habitual aplicarlo sobre el logaritmo de la variable.

En el presente trabajo, el método de Tukey se aplica sin transformación logarítmica previa sobre el precio (la única de las variables filtradas para la que el criterio estadístico resulta directamente aplicable). Se optó por no transformar el precio por dos motivos: primero, el umbral resultante (468,5 € por noche) es directamente interpretable en el dominio del problema, un significado concreto que se perdería al trabajar en escala logarítmica; segundo, los modelos de regresión empleados (árbol de decisión, bosque aleatorio y XGBoost) no asumen normalidad de los residuos, por lo que la asimetría positiva del precio no compromete la modelización.

El tamaño de la habitación y la distancia al centro urbano, en cambio, se filtran mediante umbrales absolutos fijados por el dominio del problema en lugar del criterio de Tukey, según se detalla en el Capítulo 6: se recurre a un umbral absoluto cuando el criterio estadístico de Tukey no se ajusta al dominio del problema (véase la Tabla 6.2).

3.4 Correlación lineal de Pearson

La correlación de Pearson mide el grado de asociación lineal entre dos variables aleatorias [23].

| Definición 3.5. Sean X e Y variables aleatorias con medias μ_X , μ_Y y desviaciones típicas σ_X , σ_Y respectivamente. El coeficiente de correlación de Pearson se define como

$$\rho_{X,Y} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y} = \frac{E[(X - \mu_X)(Y - \mu_Y)]}{\sigma_X \sigma_Y}.$$

El coeficiente toma valores en $[-1, 1]$. Un valor próximo a 1 indica una asociación lineal positiva fuerte (ambas variables crecen juntas), próximo a -1 indica una asociación negativa fuerte (cuando una crece la otra decrece), y próximo a 0 indica ausencia de asociación lineal (aunque puede existir asociación no lineal).

La estimación muestral del coeficiente de Pearson a partir de una muestra $\{(x_i, y_i)\}_{i=1}^n$ es

$$r_{X,Y} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}.$$

En el análisis exploratorio del conjunto de datos se calculó la matriz de correlaciones de Pearson para identificar las variables con mayor relación lineal con el precio y para detectar posibles problemas de multicolinealidad que pudieran afectar a los modelos de regresión lineal, tal como se analiza en las Secciones 4.2.2 y 6.5 más adelante.

3.5 Normalización y estandarización

Muchos algoritmos de aprendizaje automático son sensibles a la escala de las variables de entrada. La regresión lineal con regularización, los modelos basados en distancias y las redes neuronales exigen que todas las características estén en rangos comparables para que el proceso de optimización funcione correctamente. Los modelos basados en árboles (árboles de decisión, bosques aleatorios, *gradient boosting*), en cambio, son invariantes ante transformaciones monótonas de las variables en general, por lo que no requieren normalización.

3.5.1 Normalización mín-máx

| Definición 3.6. Sea $\{x_1, \dots, x_n\}$ una muestra con mínimo $x_{\min}$ y máximo $x_{\max}$. La normalización mín-máx transforma cada observación mediante

$$x'_i = \frac{x_i - x_{\min}}{x_{\max} - x_{\min}}.$$

Tras esta transformación, los valores se encuentran en el intervalo $[0, 1]$. Su inconveniente es que es muy sensible a los valores extremos: si $x_{\max}$ es un *outlier*, la mayoría de los datos quedará comprimida en una región pequeña del intervalo.

3.5.2 Estandarización *z-score*

| Definición 3.7. La estandarización o normalización *z-score* transforma cada observación mediante

$$x'_i = \frac{x_i - \bar{x}}{s},$$

donde $\bar{x}$ es la media muestral y s la desviación típica muestral.

Tras esta transformación la variable tiene media cero y desviación típica uno. Es más robusta ante *outliers* que la normalización mín-máx porque no depende de los valores extremos de la muestra.

En este trabajo se aplicó la estandarización *z-score* a las variables numéricas en los modelos para los que resulta conveniente (regresión lineal), empleando los parámetros $(\bar{x}, s)$ estimados sobre el conjunto de entrenamiento y aplicando la misma transformación al conjunto de validación y prueba para evitar filtración de información.

3.6 Codificación de variables categóricas

Los modelos de aprendizaje automático operan sobre espacios vectoriales y, por tanto, no pueden procesar directamente variables nominales como el tipo de alojamiento o la provincia. Es necesario convertirlas a una representación numérica.

3.6.1 Codificación ordinal

La codificación ordinal (*label encoding*) asigna un entero a cada categoría. Esta representación es adecuada cuando las categorías tienen un orden natural (por ejemplo, la categoría por estrellas). Cuando no existe un orden intrínseco, la codificación ordinal puede introducir una relación de orden artificial que los modelos lineales interpretarán incorrectamente.

Observación 3.1. La gravedad de esta relación de orden artificial depende del modelo. En la regresión lineal o en modelos basados en distancias, el entero asignado a cada categoría entra directamente en una combinación lineal o en un cálculo de distancia, por lo que el orden arbitrario sí distorsiona las predicciones. En los modelos basados en árboles (Sección 4.3 y siguientes), en cambio, el efecto es mucho más limitado: un árbol puede aislar cualquier subconjunto de categorías mediante particiones binarias sucesivas sobre el valor entero, a costa de necesitar más divisiones que con una codificación *one-hot*. Por ello, la codificación ordinal es una alternativa razonable a la codificación *one-hot* para variables nominales de alta cardinalidad (con muchas categorías distintas) cuando el modelo final es un árbol o un *ensemble* de árboles, como ocurre en el Capítulo 6 con la variable *localidad*.

3.6.2 Codificación *one-hot*

Definición 3.8. Sea $C = \{c_1, \dots, c_k\}$ el conjunto de valores de una variable categórica nominal. La codificación *one-hot* representa cada observación como un vector binario $\mathbf{e}_j \in \{0, 1\}^k$, donde el componente j -ésimo es 1 si la observación pertenece a la categoría c_j y 0 en caso contrario.

La codificación *one-hot* evita relaciones de orden artificiales y es la representación estándar para variables nominales en modelos lineales y en redes neuronales. Su inconveniente es que incrementa la dimensionalidad del espacio de características en $k - 1$ por cada variable categórica, lo que puede ser problemático si k es grande. En el presente trabajo se aplicó a la variable tipo de alojamiento (con dos categorías); para las variables *localidad* y *provincia*, con un número de categorías mucho mayor, se optó en cambio por la codificación ordinal, según se justifica en el Capítulo 6.

4 | Modelos de Regresión

Para predecir el precio de los alojamientos por noche, este capítulo presenta los cuatro modelos de regresión supervisada empleados. Para cada modelo se describe su formulación matemática, su proceso de entrenamiento y sus principales ventajas e inconvenientes en el contexto del problema tratado. Al final del capítulo se introducen las métricas de evaluación utilizadas para comparar el rendimiento de los modelos.

4.1 El problema de regresión

| Definición 4.1. *Un problema de regresión supervisada consiste en aprender, a partir de un conjunto de entrenamiento $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ con $\mathbf{x}_i \in \mathbb{R}^p$ y $y_i \in \mathbb{R}$, una función $f: \mathbb{R}^p \rightarrow \mathbb{R}$ tal que $f(\mathbf{x})$ sea una buena estimación de y para observaciones nuevas $\mathbf{x}$.*

En el problema concreto de este trabajo, $\mathbf{x}_i$ es el vector de características del alojamiento i (tipo, estrellas, valoración, número de reseñas, servicios, distancia al centro, provincia, localidad, indicadores temporales) e y_i es su precio por noche en euros. El objetivo es aprender f de modo que la predicción $\hat{y}_i = f(\mathbf{x}_i)$ se desvíe lo menos posible del precio real.

4.2 Regresión lineal múltiple

La regresión lineal múltiple es el modelo paramétrico más simple y, al mismo tiempo, el más interpretable. Supone que la relación entre las características y la respuesta es lineal:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p + \varepsilon = \tilde{\mathbf{x}}^\top \boldsymbol{\beta} + \varepsilon,$$

donde $\tilde{\mathbf{x}} = (1, x_1, \dots, x_p)^\top \in \mathbb{R}^{p+1}$ es el vector de características aumentado con un 1 para absorber el término independiente, $\boldsymbol{\beta} = (\beta_0, \beta_1, \dots, \beta_p)^\top$ es el vector de coeficientes y ε es el término de error, que se asume de media cero y varianza constante [20].

4.2.1 Estimación por mínimos cuadrados ordinarios

El método estándar para estimar β es el de los *mínimos cuadrados ordinarios* (OLS). Dado el conjunto de entrenamiento, se define la función de pérdida como la suma de cuadrados de los residuos:

$$\text{SSR}(\beta) = \sum_{i=1}^n (y_i - \tilde{\mathbf{x}}_i^\top \beta)^2 = \|\mathbf{y} - \mathbf{X}\beta\|^2,$$

donde $\mathbf{y} = (y_1, \dots, y_n)^\top$ y $\mathbf{X}$ es la matriz de diseño de dimensión $n \times (p+1)$ cuya fila i -ésima es $\tilde{\mathbf{x}}_i^\top = (1, x_{i1}, \dots, x_{ip})$.

| Teorema 4.1 (Estimador OLS). Si $\mathbf{X}$ tiene rango columna máximo, el mínimo de $\text{SSR}(\beta)$ es único y viene dado por

$$\hat{\beta} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}.$$

Demostración. Derivando SSR respecto a β e igualando a cero:

$$\nabla_{\beta} \text{SSR} = -2\mathbf{X}^\top (\mathbf{y} - \mathbf{X}\beta) = \mathbf{0},$$

de donde se obtienen las *ecuaciones normales* $\mathbf{X}^\top \mathbf{X}\beta = \mathbf{X}^\top \mathbf{y}$. Si $\mathbf{X}$ tiene rango columna máximo, $\mathbf{X}^\top \mathbf{X}$ es inversible y la solución es única. La condición de mínimo se verifica comprobando que la hessiana $2\mathbf{X}^\top \mathbf{X}$ es semidefinida positiva, lo que se cumple para cualquier matriz de diseño. **|**

4.2.2 Supuestos e hipótesis

La inferencia clásica sobre el estimador OLS (contrastes, intervalos de confianza) descansa en los supuestos de Gauss-Markov: linealidad, exogeneidad o media condicional cero ($E(\varepsilon_i | \mathbf{X}) = 0$, clave para la insesgadez del estimador), homoscedasticidad ($\text{Var}(\varepsilon_i) = \sigma^2$), ausencia de autocorrelación ($\text{Cov}(\varepsilon_i, \varepsilon_j) = 0$ para $i \neq j$), y ausencia de multicolinealidad exacta. Bajo estos supuestos, el estimador OLS es el mejor estimador lineal insesgado (BLUE, *Best Linear Unbiased Estimator*), según el Teorema de Gauss-Markov.

Cuando estos supuestos no se cumplen, las consecuencias varían en gravedad. Una violación especialmente relevante para la predicción es la falta de **linealidad**: si la relación real no es lineal, el modelo queda mal especificado y las predicciones presentan un sesgo sistemático que no desaparece con más datos; es lo que se observa en este trabajo, donde la regresión lineal obtiene un $R^2 = 0,14$ frente al 0,87 del bosque aleatorio (Tabla 7.3). La **heteroscedasticidad** y la **autocorrelación**, en cambio, si se mantiene la exogeneidad, no sesgan los coeficientes pero invalidan sus errores estándar, afectando a la inferencia (no a la predicción); en este trabajo cabe esperar ambas, dada la distribución asimétrica de los precios y la falta de independencia entre observaciones del mismo alojamiento en fechas distintas (Capítulo 6). La **multicolinealidad** exacta impide invertir $\mathbf{X}^\top \mathbf{X}$ (de ahí la

eliminación de una categoría en la codificación *one-hot*), y la alta colinealidad entre variables de servicios similares (Sección 3.4) desestabiliza los coeficientes individuales sin afectar gravemente a la predicción global.

4.3 Árbol de decisión para regresión

Los árboles de decisión son modelos no paramétricos que particionan el espacio de características de forma recursiva en regiones rectangular-disjuntas y asignan a cada región una predicción constante [4].

4.3.1 Algoritmo CART

El algoritmo CART (*Classification and Regression Trees*) construye el árbol mediante divisiones binarias recursivas. En cada nodo interno se elige el atributo j y el umbral t que minimizan la impureza de la partición resultante. Para regresión, la impureza de un nodo R se mide con la varianza de los valores de respuesta que caen en él:

$$\text{impureza}(R) = \frac{1}{|R|} \sum_{i \in R} (y_i - \bar{y}_R)^2,$$

donde $\bar{y}_R$ es la media de y en R . La ganancia de una división (R_L, R_R) es

$$\Delta = \text{impureza}(R) - \frac{|R_L|}{|R|} \text{impureza}(R_L) - \frac{|R_R|}{|R|} \text{impureza}(R_R).$$

El árbol crece hasta que se alcanza un criterio de parada (profundidad máxima, número mínimo de muestras en un nodo hoja, etc.). La predicción en la hoja que contiene $\mathbf{x}$ es la media de los valores de entrenamiento en esa hoja.

4.3.2 Sobreajuste y poda

Los árboles de gran profundidad tienden a memorizar el conjunto de entrenamiento (*overfitting*). La estrategia más común para controlarlo es limitar la profundidad máxima del árbol o el número mínimo de muestras requeridas para realizar una división. En el presente trabajo estos hiperparámetros se optimizan mediante búsqueda en rejilla con validación cruzada (véase la Sección 5.2).

4.4 Bosque Aleatorio

El bosque aleatorio (*Random Forest*) es un método de *ensemble* propuesto en [3] que combina múltiples árboles de decisión entrenados sobre perturbaciones

aleatorias de los datos, con el objetivo de reducir la varianza manteniendo un sesgo bajo.

4.4.1 Construcción del *ensemble*

El bosque aleatorio construye B árboles de decisión mediante el siguiente procedimiento:

1. Para cada árbol $b = 1, \dots, B$:
 - a) Se extrae una muestra *bootstrap* $\mathcal{D}_b^*$ de tamaño n con reemplazamiento de $\mathcal{D}$.
 - b) Se entrena un árbol de decisión T_b sobre $\mathcal{D}_b^*$, con la particularidad de que en cada nodo se elige la mejor división solo entre un subconjunto aleatorio de $m \leq p$ características.
2. La predicción para una nueva observación $\mathbf{x}$ se obtiene promediando las predicciones de los B árboles:

$$\hat{f}^{\text{RF}}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B T_b(\mathbf{x}).$$

La combinación del muestreo *bootstrap* (*bagging*) y la selección aleatoria de características reduce la correlación entre los árboles del *ensemble*, lo que disminuye la varianza del predictor resultante.

4.4.2 Importancia de las variables

Una de las ventajas del bosque aleatorio es que proporciona una medida natural de la importancia de cada variable. La importancia por reducción media de impureza (*Mean Decrease Impurity*) para la característica j se calcula como la reducción media de la impureza provocada por las divisiones en j a lo largo de todos los árboles:

$$\text{Imp}(j) = \frac{1}{B} \sum_{b=1}^B \sum_{\substack{v \in T_b \\ \text{divide en } j}} \frac{|R_v|}{n} \Delta_v,$$

donde la suma interior recorre todos los nodos v del árbol T_b que dividen sobre la característica j , y Δ_v es la reducción del error cuadrático medio en ese nodo (al tratarse de árboles de regresión, la impureza se mide mediante varianza y no mediante el índice de Gini, propio de árboles de clasificación). Esta medida es especialmente útil para interpretar qué atributos del alojamiento son más determinantes para el precio.

4.4.3 Hiperparámetros

Los hiperparámetros principales del bosque aleatorio son el número de árboles B , la profundidad máxima de cada árbol, el número de características a considerar en cada división m (habitualmente $m = \lfloor \sqrt{p} \rfloor$ en clasificación y $m = \lfloor p/3 \rfloor$ en regresión), y el tamaño mínimo de los nodos hoja. En el presente trabajo se optimizan mediante búsqueda en rejilla con validación cruzada.

4.5 Gradient Boosting y XGBoost

El *gradient boosting* es otra técnica de *ensemble* que construye el predictor de forma aditiva e iterativa. A diferencia del bosque aleatorio, donde los árboles se construyen de forma independiente y paralela, en el *gradient boosting* cada árbol se entrena para ajustar los errores del predictor acumulado hasta ese momento [11].

4.5.1 Formulación general

El objetivo es minimizar una función de pérdida $\mathcal{L}(y, F(\mathbf{x}))$ respecto al predictor F . El algoritmo inicializa $F_0(\mathbf{x}) = \arg \min_c \sum_i \mathcal{L}(y_i, c)$ y en cada iteración m realiza los siguientes pasos:

1. Calcula los *pseudoresiduos*:

$$r_{im} = - \left[\frac{\partial \mathcal{L}(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)} \right]_{F=F_{m-1}}.$$

2. Ajusta un árbol de regresión h_m sobre los pseudoresiduos $\{(\mathbf{x}_i, r_{im})\}_{i=1}^n$.
3. Actualiza el predictor:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta h_m(\mathbf{x}),$$

donde $\eta \in (0, 1]$ es la tasa de aprendizaje (*learning rate*).

Para regresión con pérdida cuadrática $\mathcal{L}(y, \hat{y}) = (y - \hat{y})^2/2$, los pseudoresiduos coinciden con los residuos ordinarios $r_{im} = y_i - F_{m-1}(\mathbf{x}_i)$.

4.5.2 XGBoost

XGBoost (*eXtreme Gradient Boosting*) [9] es una implementación optimizada del *gradient boosting* que añade varias mejoras respecto al algoritmo estándar:

- **Regularización:** incorpora términos de regularización ℓ_1 y ℓ_2 sobre los pesos de las hojas en la función objetivo, reduciendo el sobreajuste.
- **División aproximada:** en lugar de evaluar todos los puntos de corte posibles, utiliza cuantiles de la distribución de cada característica para reducir el coste computacional.
- **Paralelización:** la construcción de cada árbol se paraleliza a nivel de nodo, lo que lo hace significativamente más rápido que implementaciones previas.
- **Manejo de valores nulos:** XGBoost aprende automáticamente la dirección óptima para las observaciones con valores faltantes en cada nodo.

La función objetivo de XGBoost en la iteración m es

$$\mathcal{L}^{(m)} = \sum_{i=1}^n \ell(y_i, \hat{y}_i^{(m-1)} + h_m(\mathbf{x}_i)) + \Omega(h_m),$$

donde $\Omega(h) = \gamma T + \frac{1}{2} \lambda \|\mathbf{w}\|^2$ penaliza el número de hojas T y la norma de los pesos de hoja $\mathbf{w}$.

4.6 Métricas de evaluación para regresión

Para comparar el rendimiento de los modelos de regresión se emplean tres métricas complementarias [12].

| Definición 4.2 (Error cuadrático medio).

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2.$$

Su raíz, $\text{RMSE} = \sqrt{\text{MSE}}$, tiene las mismas unidades que la variable respuesta (euros) y es una métrica ampliamente usada en problemas de regresión.

| Definición 4.3 (Error absoluto medio).

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|.$$

El MAE es más robusto que el RMSE ante outliers de la variable respuesta, pues no eleva las diferencias al cuadrado.

| Definición 4.4 (Coeficiente de determinación).

$$R^2 = 1 - \frac{\sum_{i=1}^n (\hat{y}_i - y_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}, \quad \bar{y} = \frac{1}{n} \sum_{i=1}^n y_i,$$

donde $\bar{y}$ es la media muestral de la variable respuesta. El R^2 toma valores en $(-\infty, 1]$, donde $R^2 = 1$ indica predicción perfecta y $R^2 = 0$ equivale a predecir siempre la media de la muestra.

El R^2 mide la proporción de varianza de la respuesta explicada por el modelo, lo que lo hace especialmente interpretable. Sin embargo, en modelos complejos tiende a sobreestimar el ajuste si se calcula sobre los mismos datos usados para entrenar; por eso se evalúa siempre sobre los conjuntos de validación y prueba, reservados y no utilizados durante el entrenamiento.

Observación 4.1. En el problema de predicción de precios de alojamiento, un MAE de k euros significa que el modelo se equivoca en promedio absoluto k euros en la estimación del precio por noche. Esta interpretación directa es la razón por la que el MAE se reporta como referencia práctica del error en euros; no obstante, es el R^2 el criterio empleado en este trabajo para comparar y seleccionar entre los cuatro modelos candidatos (Capítulo 7), al resumir en un único valor el poder explicativo global de cada uno.

5 | Validación de Modelos y Categorización de Chollos

Para empezar, este capítulo describe los procedimientos de validación empleados para comparar y seleccionar el mejor modelo de regresión (entrenado y evaluado con las métricas presentadas en la Sección 4.6): la partición de datos y la búsqueda de hiperparámetros mediante validación cruzada. Después, se explica la categorización determinista de chollos a partir de la comparación entre el precio real y el precio predicho por el modelo de regresión, justificando por qué se optó por esta estrategia en lugar de un clasificador de aprendizaje automático.

5.1 Partición del conjunto de datos

Para evaluar correctamente los modelos se adoptó una partición en tres conjuntos:

- **Entrenamiento** (70 %): datos sobre los que se ajusta el modelo y se ejecuta la búsqueda de hiperparámetros mediante validación cruzada.
- **Validación** (15 %): datos utilizados para comparar modelos y tomar decisiones de selección.
- **Prueba** (15 %): datos reservados y no utilizados en ninguna fase de entrenamiento ni selección. Se emplean exclusivamente para la evaluación final del modelo seleccionado, una única vez.

La elección de las proporciones 70/15/15 responde al tamaño del conjunto de datos disponible. Cuando el volumen de datos es limitado, es habitual reservar una proporción mayor para validación y prueba (por ejemplo, 60/20/20) para compensar la incertidumbre de estimar el rendimiento con pocas observaciones. Sin embargo, con un conjunto de más de 2,6 millones de registros, incluso el 15 % reservado para validación y para prueba supone cientos de miles de observaciones cada uno, más que suficientes para obtener estimaciones estables y de baja varianza tanto en la selección de hiperparámetros como en la evaluación final. Esto permite destinar la mayor parte de los datos (70 %) al entrenamiento, lo que favorece

especialmente a los modelos de conjunto como el bosque aleatorio, que se benefician de un mayor volumen de datos para reducir la varianza de sus estimaciones. Además, dividir a partes iguales el 30 % restante entre validación y prueba asegura que ambos conjuntos tengan un tamaño comparable, de modo que la fiabilidad de la selección de hiperparámetros (sobre validación) y la de la evaluación final (sobre prueba) sea equivalente.

La existencia de un conjunto de validación independiente, además de entrenamiento y prueba, responde a una necesidad concreta: decidir cuál de los cuatro modelos candidatos es el mejor sin comprometer la evaluación final. Si esa decisión se tomara consultando el rendimiento sobre el conjunto de prueba, la métrica final reportada dejaría de ser una estimación honesta del comportamiento del modelo sobre datos nunca vistos, precisamente el problema que esta partición en tres conjuntos evita.

La partición se implementó en dos pasos sucesivos para garantizar proporciones exactas:

1. División 70-30 entre entrenamiento (1.822.153 registros, 70,000 %) y un conjunto provisional (780.923 registros).
2. División 50-50 del conjunto provisional entre validación (390.461 registros, 15,000 %) y prueba (390.462 registros, 15,000 %).

Esta estrategia garantiza que los resultados reportados en el Capítulo 7 sean estimaciones honestas del rendimiento del modelo sobre datos no vistos, evitando el sobreajuste que ocurre cuando se toma alguna decisión de modelización en función del conjunto de prueba.

5.2 Validación cruzada y búsqueda de hiperparámetros

Los modelos de aprendizaje automático descritos en el Capítulo 4 contienen hiperparámetros que no se aprenden de los datos sino que deben fijarse antes del entrenamiento. La elección de estos hiperparámetros influye de forma determinante en el rendimiento del modelo.

5.2.1 Validación cruzada de k pliegues

| Definición 5.1 (k -fold cross-validation). Sea $\mathcal{D}$ el conjunto de entrenamiento de tamaño n . La validación cruzada de k pliegues divide $\mathcal{D}$ aleatoriamente en k subconjuntos (pliegues) de tamaño aproximadamente igual. Para cada pliegue j :

1. Se entrena el modelo sobre los $k - 1$ pliegues restantes.
2. Se evalúa el modelo sobre el pliegue j , obteniendo la puntuación s_j .

La estimación del error generalizado es $\bar{s} = \frac{1}{k} \sum_{j=1}^k s_j$.

En este trabajo se utilizó $k = 3$ como balance entre costo computacional y calidad de la estimación, dada la elevada dimensionalidad del espacio de hiperparámetros a explorar y el alto volumen del conjunto de datos.

5.2.2 Búsqueda en rejilla

La *búsqueda en rejilla con validación cruzada* (*GridSearchCV*) evalúa todas las combinaciones posibles de una rejilla de valores candidatos para cada hiperparámetro, seleccionando la configuración que minimiza el error cuadrático medio negativo en la validación cruzada [24]. La función de puntuación empleada fue `neg_mean_squared_error`, de forma que la combinación óptima minimiza el RMSE medio sobre los 3 pliegues.

La rejilla de hiperparámetros explorada para cada modelo se detalla en la Tabla 7.2 del Capítulo 7. El parámetro `n_jobs=-1` se estableció en todos los modelos para aprovechar el paralelismo de los múltiples núcleos del procesador durante la búsqueda.

5.2.3 Estandarización y filtración de información

Para la regresión lineal, que es sensible a la escala de las variables de entrada, se aplicó estandarización *z-score* (véase la Sección 3.5.2) exclusivamente sobre el conjunto de entrenamiento:

- El escalador (`StandardScaler`) aprende media y desviación típica sobre el conjunto de entrenamiento: `fit_transform(X_train)`.
- Los conjuntos de validación y prueba se transforman con los mismos parámetros aprendidos: `transform(X_val)`, `transform(X_test)`.

Esta distinción es crucial para evitar la *filtración de información* (*data leakage*): si el escalador aprendiera los parámetros sobre todos los datos incluyendo el conjunto de prueba, el modelo dispondría implícitamente de información sobre datos futuros, produciendo estimaciones de rendimiento optimistamente sesgadas.

Los modelos basados en árboles (árbol de decisión, bosque aleatorio, XGBoost) no requieren estandarización al ser invariantes ante transformaciones monótonas de las variables.

5.3 Categorización determinista de chollos

Una vez entrenado y seleccionado el modelo de regresión ganador, el sistema compara el precio real de cada alojamiento y con el precio predicho $\hat{y}$ para asignarle una etiqueta de categoría. La comparación se realiza mediante el cociente:

$$r = \frac{y}{\hat{y}}.$$

Un valor $r < 1$ indica que el precio real es inferior al precio que el modelo estima como justo para ese alojamiento (potencial chollo), mientras que $r > 1$ indica que el precio real supera al estimado (alojamiento inflado).

Las cinco categorías y sus umbrales son:

Cuadro 5.1: Categorías de chollo según el cociente r entre precio real y precio predicho.

Código	Etiqueta	Condición	Interpretación
4	Hiper-chollo	$r \leq 0,75$	Al menos un 25 % por debajo del justo
3	Super-chollo	$0,75 < r < 0,85$	Entre 15 % y 25 % por debajo
2	Chollo	$0,85 \leq r < 0,97$	Entre 3 % y 15 % por debajo
1	Normal	$0,97 \leq r \leq 1,03$	Precio dentro del ± 3 %
0	Inflado	$r > 1,03$	Precio real > 3 % por encima del justo

La implementación utiliza `numpy.select` para aplicar las condiciones de forma vectorizada sobre el conjunto completo sin bucles explícitos, lo que es esencial para el rendimiento cuando el *dataset* contiene millones de registros.

5.3.1 Justificación: categorización determinista frente a clasificador ML

Una alternativa natural sería entrenar un clasificador de aprendizaje automático (regresión logística, árbol de decisión, etc.) sobre las etiquetas de categoría. Sin embargo, esta aproximación presenta dos inconvenientes fundamentales que llevaron a descartarla en favor de la regla determinista:

1. **Redundancia conceptual.** La categoría chollo no es una propiedad intrínseca del alojamiento observable en los datos de entrada, sino una *relación* entre el precio real y la predicción del modelo de regresión. Entrenar un clasificador sobre etiquetas derivadas del propio modelo de regresión añadiría una capa de aprendizaje innecesaria sobre información que ya está completamente contenida en el ratio r .

2. **Interpretabilidad y reproducibilidad.** La regla determinista basada en umbrales del ratio es completamente transparente: dado un precio real y un precio predicho, cualquier usuario puede verificar manualmente la categoría asignada. Un clasificador de caja negra añadiría complejidad además de una fuente adicional de error (el error de clasificación) sobre el error de regresión ya existente.

Observación 5.1. Los umbrales del ratio (0,75, 0,85, 0,97, 1,03) se fijaron atendiendo a criterios de utilidad práctica para el usuario final: un margen del $\pm 3\%$ para la categoría «Normal» refleja la variación de precio que podría atribuirse a ruido de medición o pequeñas fluctuaciones de demanda, mientras que el umbral del 25 % para «Hiper-chollo» corresponde a descuentos suficientemente llamativos como para representar una oportunidad de ahorro relevante.

Parte III

Desarrollo Práctico

6 | Sistema de Extracción y Preprocesamiento de Datos

La fase práctica de obtención y preparación de los datos es el objeto de este capítulo. Se describe la fuente y recopilación de los datos, el protocolo de limpieza y transformación, la ingeniería de características aplicada y las estadísticas descriptivas del conjunto de datos resultante.

6.1 Fuente y recopilación de los datos

Los datos empleados en este trabajo fueron obtenidos de una plataforma de reserva de alojamientos turísticos de amplia difusión y cobertura internacional, que agrega millones de listados de distintas tipologías (hoteles, apartamentos, estudios, casas, etc.) en todo el mundo. Por prudencia y dado el carácter académico del trabajo, el nombre de la misma no se hace explícito en esta memoria; en todo caso, los datos se utilizan con fines exclusivamente académicos y de investigación. La metodología descrita no depende de las particularidades de esta plataforma en concreto, y se detalla al nivel necesario para poder reproducirse sobre una plataforma equivalente.

La recopilación se centró en los alojamientos disponibles en las ocho provincias de la comunidad autónoma de Andalucía: Almería, Cádiz, Córdoba, Granada, Huelva, Jaén, Málaga y Sevilla, incluyendo un horizonte de 365 días de precios por alojamiento, con el objetivo de obtener la mayor cobertura posible del mercado.

El proceso se organizó en dos etapas, correspondientes a las dos fases de información disponibles en la plataforma: un listado general de resultados por provincia y una ficha individual por cada alojamiento. En la primera etapa, apoyada en Selenium [28], se obtuvo de cada resultado: localización de búsqueda, título del alojamiento, URL de la ficha, valoración media de los clientes, número de valoraciones, tipo de alojamiento y categoría por estrellas.

En la segunda etapa, apoyada en Playwright [19], se accedió a la API interna de la plataforma para obtener los servicios disponibles y el calendario de precios (mismo horizonte de 365 días) de cada alojamiento. Un módulo auxiliar complementó

esta etapa extrayendo el tamaño real de la habitación más barata disponible. Los campos obtenidos en esta fase son: identificador del hotel, coordenadas GPS (latitud y longitud), servicios generales del establecimiento, servicios de la habitación más barata, tamaño de la habitación (m²) y calendario de precios diarios.

La implementación completa del proceso de recopilación puede consultarse en el repositorio del proyecto [16].

6.1.1 Geocodificación de los centros urbanos

Para calcular la distancia de cada alojamiento al centro de su localidad, se obtuvieron las coordenadas geográficas del centro de todas las localidades presentes en el conjunto de datos mediante la API de Nominatim [22] del proyecto OpenStreetMap. La distancia se calculó mediante la fórmula del haversine vectorizada:

$$d = 2r \arcsin \sqrt{\sin^2\left(\frac{\Delta\phi}{2}\right) + \cos\phi_1 \cos\phi_2 \sin^2\left(\frac{\Delta\lambda}{2}\right)},$$

donde $r \approx 6371$ km es el radio medio de la Tierra; (ϕ_1, λ_1) y (ϕ_2, λ_2) son la latitud y longitud (en radianes) del alojamiento y del centro urbano de referencia, respectivamente; y $\Delta\phi = \phi_2 - \phi_1$, $\Delta\lambda = \lambda_2 - \lambda_1$.

6.2 Preprocesamiento de los datos

Los datos en bruto obtenidos en la etapa de recopilación contienen inevitablemente ruido, valores nulos e inconsistencias que es necesario corregir antes de entrenar los modelos. El protocolo de preprocesamiento se compone de las siguientes fases.

6.2.1 Tratamiento de valores nulos

El tratamiento de valores nulos se divide en dos estrategias complementarias en función de la naturaleza de la variable afectada:

- **Eliminación de registros (dropna).** Las variables de coordenadas geográficas (latitud, longitud, latitud del centro y distancia al centro) son imprescindibles para calcular la variable `distancia_centro_km`, que el análisis de correlación identifica como uno de los predictores más relevantes del precio. Los registros sin esta información (un 2,2 % del total) se eliminan directamente, pues no hay manera de recuperarla sin introducir sesgos. Adicionalmente, se eliminaron registros con localidad codificada como '9', correspondiente a un error de geocodificación detectado manualmente.

- **Imputación (fillna).** Para el resto de variables con valores nulos se aplica la estrategia de imputación detallada en la Tabla 6.1, priorizando estadísticos robustos que minimicen el sesgo introducido.

Cuadro 6.1: Estrategia de imputación de valores nulos por variable.

Variable	Valor imputado	Justificación
codigo_postal	-1	Desconocido; no se usa en el modelo
tipo	'Otro'	Sin tipo definido → categoría residual
estrellas	1	Sin clasificación → mínimo (no hotel)
n_valoraciones	0	Sin reseñas todavía (alojamiento nuevo)
valoracion_clientes	media	Variable acotada (0-10) sin outliers extremos; la media representa bien la tendencia central
tamaño_habitacion	mediana	Más robusta que la media ante suites con m ² extremos

6.2.2 Detección y eliminación de valores atípicos

Se aplicó el método de Tukey (Sección 3.3) y umbrales fijos a las variables numéricas más relevantes, con criterios específicos justificados por el análisis exploratorio de los datos. El filtrado se restringió a las tres variables en las que un valor extremo indica plausiblemente un error de captura (de extracción o de geocodificación) o un alojamiento ajeno al segmento de estudio: precio, tamaño de la habitación y distancia al centro urbano. El resto de variables numéricas (número de reseñas, días restantes hasta la estancia) presentan colas largas que reflejan variabilidad genuina del fenómeno observado y no un error de medición, por lo que se mantuvieron sin filtrar en esta etapa.

Precio por noche

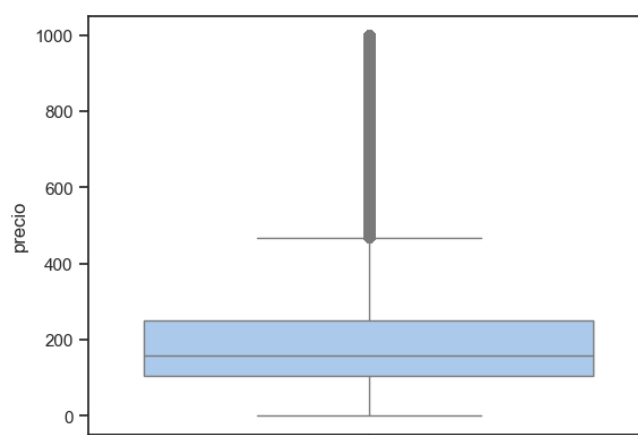


Figura 6.1: Diagrama de caja del precio por noche antes de la eliminación de valores atípicos.

Nota: el bigote superior se sitúa en torno al umbral de Tukey (468,5 €), por encima del cual se concentra una larga cola de alojamientos de precio elevado considerados valores atípicos.

El criterio de Tukey sitúa el límite superior en 468,5 €/noche. Se optó por eliminar los registros con precio superior a este umbral por dos razones: los alojamientos con precios tan elevados (villas de lujo, suites de gran hotel) son excepcionalmente raros y representan un segmento de mercado ajeno al perfil de usuario de la aplicación; además, su presencia distorsionaría el modelo al introducir rangos de precio muy alejados del comportamiento general.

Tamaño de la habitación

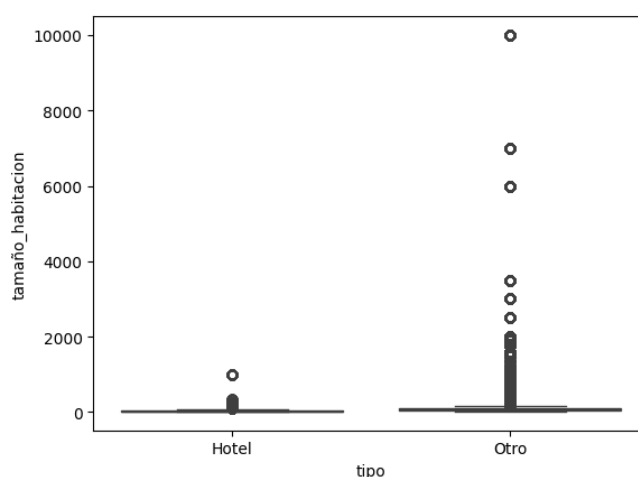


Figura 6.2: Diagrama de caja del tamaño de la habitación por tipo de alojamiento antes de la eliminación de valores atípicos.

Nota: la categoría «Otro» concentra los valores atípicos más extremos (hasta 10.000 m²), correspondientes a casas rurales y fincas que el umbral de 150 m² excluye del conjunto de datos final.

Se fijó un umbral de 150m^2 , que elimina el 7 % de los registros (191.738 registros). La motivación no es puramente estadística sino de homogeneidad del conjunto de datos: las propiedades con tamaño superior corresponden mayoritariamente a casas rurales y fincas, cuyas dinámicas de precio son estructuralmente distintas a las del alojamiento turístico urbano y semi-urbano. En particular, la variable `distancia_centro_km` actúa de forma opuesta para este tipo de propiedades (estar lejos del centro es parte de su atractivo, no un penalizador de precio), y el tamaño refleja la superficie total de la propiedad, no la de una habitación individual. Mantenerlas en el *dataset* forzaría al modelo a aprender un único patrón para dos segmentos de mercado con lógicas de precio incompatibles. Restringir el análisis exclusivamente a hoteles habría resuelto el problema de heterogeneidad, pero los hoteles representan únicamente el 8 % de los registros; el umbral de 150m^2 permite conservar hoteles, apartamentos y estudios (el alojamiento turístico típico) con un coste de solo el 7 % de registros eliminados. El valor concreto de 150m^2 se eligió por ser el límite natural que engloba los apartamentos y suites de mayor tamaño dentro del mercado de alojamiento urbano, sin llegar a incluir propiedades de tipo rural.

Distancia al centro urbano

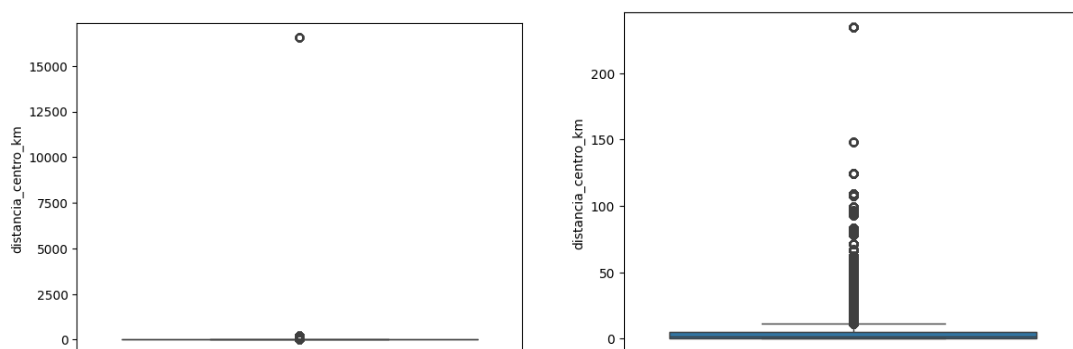


Figura 6.3: Diagrama de caja de la distancia al centro urbano antes (izquierda) y después (derecha) de eliminar el error de geocodificación de Guadalcanal.

Nota: el punto aislado en torno a 17.500 km en el diagrama izquierdo corresponde al alojamiento geolocalizado erróneamente en las Islas Salomón; al eliminarlo se recupera una escala interpretable sobre la que aplicar el umbral definitivo de 15 km.

Esta variable requirió un tratamiento en dos pasos:

1. Eliminación de registros con distancia superior a 15.000 km, que correspondían a un error de geocodificación: un alojamiento en la localidad de Guadalcanal (Sevilla) había sido geolocalizado erróneamente en las Islas Salomón.
2. Umbral definitivo de 15 km: alojamientos situados a más de 15 km del centro urbano de referencia se consideran probables errores de geocodificación o fincas rurales fuera del alcance del estudio.

La Tabla 6.2 resume el impacto de cada filtro sobre el conjunto de datos.

Cuadro 6.2: Registros eliminados en cada etapa del filtrado de valores atípicos.

Filtro	Registros eliminados	% del total
Precio > 468,5 € (Tukey)	245.170	8 %
Tamaño > 150 m ²	191.738	7 %
Distancia > 15 km	83.540	3 %
Total eliminado	520.448	17 %

6.2.3 Extracción de servicios

Cada ficha de alojamiento incluye una lista de servicios disponibles obtenida mediante la API GraphQL. A partir de esta información se generaron 12 variables binarias (int8), una por servicio de interés:

Variable	Descripción
Parking	Aparcamiento (de pago o gratuito)
Parking_gratis	Aparcamiento gratuito específicamente
Gimnasio	Sala de fitness o gimnasio
Restaurante	Restaurante en el establecimiento
Piscina	Piscina (cualquier tipo)
Piscina_interior	Piscina interior climatizada
Piscina_infinita	Piscina infinita (<i>infinity pool</i>)
Aire	Aire acondicionado en las habitaciones
Calefaccion	Calefacción en las habitaciones
Vistas	Buenas vistas en la habitación
Terraza	Terraza o balcón privado
Bano_privado	Baño privado en la habitación

La extracción se realizó mediante búsqueda de subcadenas en los campos de texto de servicios devueltos por la API, tras normalización a minúsculas.

6.3 Ingeniería de características

6.3.1 Variables temporales

Del calendario de precios de cada alojamiento se extrae, para cada precio, la fecha de la estancia. A partir de ella se calcularon cinco indicadores:

- `es_finde (int8)`: vale 1 si la fecha cae en viernes o sábado.
- `es_domingo (int8)`: vale 1 si la fecha es domingo, que muestra un comportamiento de precio diferenciado.
- `dias_restantes (int16)`: número de días hasta la fecha de la estancia desde la fecha de extracción, que captura el efecto de la anticipación de la reserva.
- `mes_disponible`: mes de la fecha de estancia, que captura la estacionalidad.
- `dia_disponible`: día del mes de la fecha de estancia.

6.3.2 Codificación de variables categóricas

La variable `tipo` (dos categorías efectivas tras la limpieza: Hotel y Otro) se codificó con *one-hot encoding* sin eliminación automática (`drop_first=False`), eliminando a continuación manualmente la columna `tipo_Otro` para quedarse con la variable binaria `tipo_Hotel` (vale 1 si el alojamiento es un hotel, 0 en caso contrario), cuya interpretación es más directa.

Las variables `localidad` (varios centenares de categorías) y `provincia` (8 categorías) se codificaron con *label encoding*, asignando un entero único a cada valor. Se optó por este enfoque en lugar del *one-hot encoding* para evitar el aumento excesivo de la dimensionalidad del espacio de características que habría generado la codificación de la localidad: con varios centenares de localidades distintas, una codificación *one-hot* habría añadido varios centenares de columnas binarias en su mayoría con un único valor positivo, encareciendo el entrenamiento sin aportar información adicional al modelo.

Como se discute en el Capítulo 3 (Observación sobre la codificación ordinal), esta decisión es viable precisamente porque el modelo ganador, el bosque aleatorio, es un *ensemble* de árboles: el orden artificial que el *label encoding* impone sobre las localidades no distorsiona la predicción del mismo modo que lo haría en una regresión lineal, ya que el árbol puede aislar cualquier localidad o grupo de localidades mediante particiones binarias sucesivas sobre su valor entero, sin necesidad de que ese valor tenga un significado de orden real. Esta misma codificación se reutilizó para la regresión lineal por consistencia entre modelos, lo que explica en parte su rendimiento sensiblemente inferior (Capítulo 7).

Los codificadores (`LabelEncoder`) se serializan con `joblib` para garantizar que la aplicación web aplica exactamente el mismo mapeo que el modelo vio durante el entrenamiento.

6.4 Estadísticas descriptivas del conjunto de datos

Tras el preprocesamiento, el conjunto de datos final contiene **2.603.076 registros**, resultado de expandir las fichas de alojamientos únicos por el calendario de

365 días de precios de cada uno. Este diseño permite al modelo capturar la variabilidad temporal del precio (efecto de la fecha, la anticipación y la estacionalidad), lo que introduce cierta dependencia entre las filas de un mismo alojamiento, tal como se discute en la Sección 1.3 (Alcance y limitaciones).

La eficiencia del formato de almacenamiento queda patente en la comparación entre los datos en bruto y los datos procesados: el fichero CSV crudo de solo una provincia (Málaga) contiene 8.871 registros en 25 columnas y ocupa 279,7 MB en disco, mientras que el fichero Parquet procesado con las ocho provincias y el calendario completo contiene 2.603.076 registros en 33 columnas y ocupa únicamente 11,2 MB. Esta reducción de **25**× en tamaño con **312**× más filas se debe a la compresión columnar del formato Parquet, al tipado óptimo de las variables (`int8`, `float32`, `category`) y a la limpieza realizada durante el preprocesamiento.

Las Figuras 6.4 y 6.5 muestran respectivamente una muestra de los datos en bruto y del conjunto procesado final, ilustrando tanto la estructura de columnas como la diferencia en dimensiones y peso en disco.

lugar	título	tipo	estrellas	valoracion_cli...	n_valoraciones	servicios
Málaga	Ona Las Rampas	Hotel	3	8,4	4042	["Accesible para personas sord...
Málaga	First Flatotel International	Hotel	3	7,7	4006	["Acceso a pisos superiores en...
Málaga	Nch Hotel Torremolinos	Hotel	2	7,7	3497	["Aire acondicionado", "Armari...
Málaga	Estudio céntrico en Torremolin...	Apartamento/Casa/Estudio	3	9,2	12	["Acceso a pisos superiores en...
Málaga	APARTAMENTOS Los PECES MALAGA	Apartamento/Casa/Estudio	4	9,5	86	["Accesible para personas sord...

Figura 6.4: Muestra de los datos en bruto de Málaga

provincia	localidad	tipo	estrellas	distancia_cent...	tamaño_habitac...	precio
Huelva	Huelva	Hotel	3	0.45	15	108
Huelva	Huelva	Hotel	3	0.45	15	50
Huelva	Huelva	Hotel	3	0.45	15	72
Huelva	Huelva	Hotel	3	0.45	15	63
Huelva	Huelva	Hotel	3	0.45	15	63

Figura 6.5: Muestra del conjunto procesado total (incluye las ocho provincias)

La Tabla 6.3 recoge los principales estadísticos descriptivos de las variables continuas del conjunto de datos final.

Cuadro 6.3: Estadísticos descriptivos de las variables continuas del conjunto de datos.

Variable	Media	Desv. típica	Mín	Q_1	Mediana	Q_3	Máx
Precio (€/noche)	167,6	90,8	1,0	102,0	147,0	213,0	468,0
Valoración (0–10)	8,6	1,0	1,0	8,2	8,7	9,2	10,0
Número de valoraciones	263	818	0	6	29	103	19.574
Estrellas	3,1	1,0	1,0	3,0	3,0	4,0	5,0
Tamaño habitación (m ²)	64,3	30,4	15,0	42,0	60,0	80,0	150,0
Distancia centro (km)	2,6	3,0	0,0	0,4	1,1	4,4	15,0
Días restantes ^a	154,3	94,1	-1,0	92,0	140,0	219,0	364,0

a. El valor mínimo de -1 en «Días restantes» afecta al 0,026 % de los registros (674 de 2.603.076), confinados a tres provincias, y se debe a un desfase de un día en la fecha de extracción asignada manualmente por provincia durante el preprocesamiento.

La distribución de alojamientos por provincia muestra una concentración notable en Málaga (37,7 % del total), seguida de Cádiz (15,8 %), Sevilla (13,5 %) y Granada (12,9 %), lo que refleja el peso diferencial de estas provincias en el turismo andaluz. Las provincias de Córdoba (7,1 %), Almería (6,5 %), Jaén (3,3 %) y Huelva (3,1 %) contribuyen con porcentajes menores.

La distribución de precios presenta una cola derecha larga, con valores comprendidos entre pocos euros y el umbral de 468,5 € establecido durante el preprocesamiento. La asimetría positiva observada es consistente con una distribución log-normal, comportamiento habitual en variables de precio en mercados de alojamiento turístico.

El número de valoraciones presenta una asimetría todavía más pronunciada (media de 263 frente a una mediana de 29), reflejo de que la mayoría de los alojamientos acumulan un número modesto de reseñas mientras que unos pocos establecimientos muy consolidados concentran varios miles.

6.5 Análisis de correlaciones

Para identificar las variables con mayor relación lineal con el precio y detectar posibles problemas de multicolinealidad, se calculó la matriz de correlaciones de Pearson (véase la Sección 3.4) sobre el conjunto de datos codificado, que comprende 26 variables. El cálculo y la visualización se realizaron en R 4.x mediante la librería `corrplot` [34], usando el formato `Apache Arrow Parquet` como interfaz de intercambio entre Python y R.

La conclusión más relevante del análisis es que **ninguna variable presenta una correlación lineal fuerte con el precio**: todas las celdas de la fila `precio` son de color muy claro, lo que indica correlaciones de Pearson próximas a cero.

La variable con mayor correlación lineal es `tamaño_habitacion` ($p=0.27$). En un escalón más abajo se encontrarían, `distancia_centro_km` ($p=0.17$), `provincia` ($p=0.15$), `estrellas` ($p=0.15$) y `Piscina` ($p=0.15$), aunque todas ellas con valores bajos. Este resultado sugiere que las relaciones entre los predictores y el precio son fundamentalmente **no lineales**, lo que explica la diferencia tan pronunciada entre el coeficiente de determinación de la regresión lineal ($R^2 = 0,14$) y el del bosque aleatorio ($R^2 = 0,87$), como se confirma en el Capítulo 7: este último es capaz de capturar esas relaciones no lineales que la regresión lineal no puede modelar.

Entre las variables predictoras destaca la alta correlación entre `Parking` y `Parking_gratis`, esperable dado que la segunda implica la primera. El resto de variables de servicios presentan correlaciones bajas entre sí, lo que sugiere que los servicios disponibles en los alojamientos tienden a ser independientes unos de otros en el conjunto de datos. Como se explica en la Sección 4.2.2, este tipo de colinealidad alta pero no exacta desestabiliza los coeficientes individuales sin afectar gravemente a la predicción global del modelo.

La Figura 6.6 muestra la matriz de correlaciones de Pearson completa calculada sobre el *dataset* codificado.

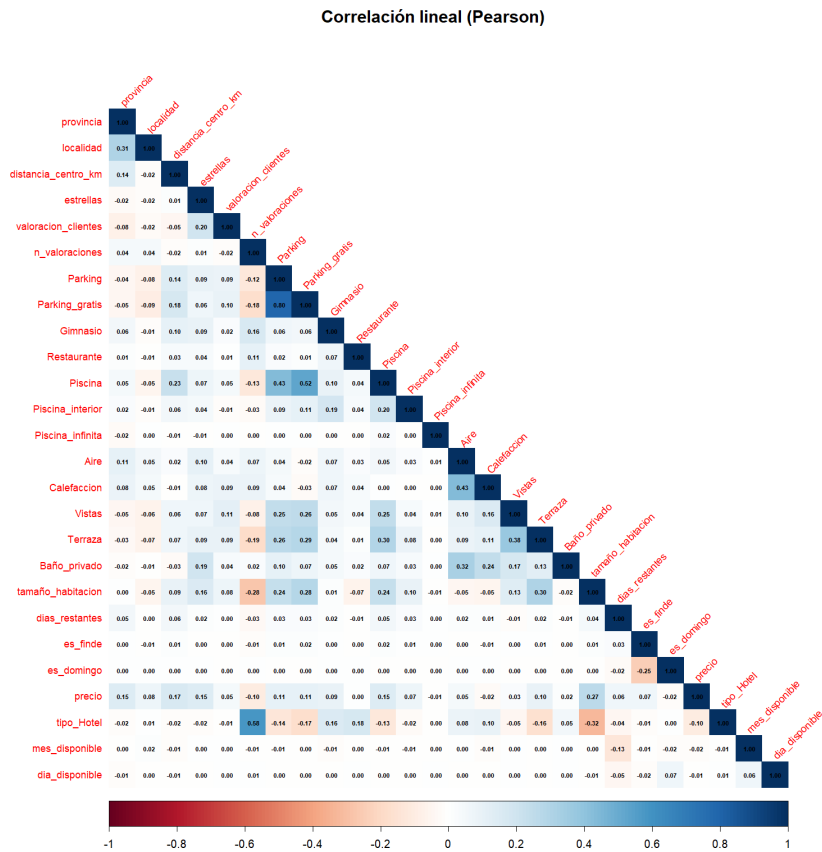


Figura 6.6: Matriz de correlaciones de Pearson sobre el *dataset* codificado (26 variables).

Nota: escala de color: azul oscuro = correlación positiva fuerte ($\rho \approx 1$), rojo oscuro = correlación negativa fuerte ($\rho \approx -1$). Generada con la librería `corrplot` de R 4.x [34].

7 | Modelización, Resultados y Aplicación Web

Los resultados de los experimentos de modelización, junto con la aplicación web desarrollada para el despliegue del sistema, se presentan a continuación. En primer lugar se describe el diseño experimental: la partición de los datos y los hiperparámetros candidatos. Después se muestran los resultados de los modelos de regresión y la distribución de chollos obtenida. Por último se describe la arquitectura de la aplicación Streamlit. El capítulo concluye con las conclusiones del trabajo y las líneas de trabajo futuro.

7.1 Diseño experimental

7.1.1 Partición y tamaño de los conjuntos

El conjunto de datos de 2.603.076 registros descrito en el Capítulo 6 se dividió de forma aleatoria con semilla fija (`random_state=42`) en tres conjuntos de acuerdo con el esquema de la Sección 5.1. La Tabla 7.1 muestra el tamaño de cada partición.

Cuadro 7.1: Tamaño de las particiones del conjunto de datos.

Conjunto	Registros	Porcentaje
Entrenamiento	1.822.153	70 %
Validación	390.461	15 %
Prueba	390.462	15 %
Total	2.603.076	100 %

7.1.2 Vector de características

Tras el preprocesamiento e ingeniería de características, el vector de entrada $\mathbf{x} \in \mathbb{R}^p$ contiene:

- **Características del alojamiento:** `tipo_Hotel` (OHE), `estrellas`, `valoracion_clientes`, `n_valoraciones`, `tamaño_habitacion`, `distancia_centro_km`.
- **Servicios:** 12 variables binarias (`Parking`, `Parking_gratis`, `Gimnasio`, `Restaurante`, `Piscina`, `Piscina_interior`, `Piscina_infinita`, `Aire`, `Calefaccion`, `Vistas`, `Terraza`, `Bano_privado`).
- **Localización:** `provincia` y `localidad` (codificación ordinal).
- **Temporales:** `es_finde`, `es_domingo`, `dias_restantes`, `mes_disponible`, `dia_disponible`.

7.1.3 Rejilla de hiperparámetros

Los hiperparámetros explorados mediante búsqueda en rejilla con validación cruzada de 3 pliegues (`scoring='neg_mean_squared_error'`) se recogen en la Tabla 7.2.

Cuadro 7.2: Rejilla de hiperparámetros para cada modelo de regresión.

Modelo	Hiperparámetro	Valores candidatos
Regresión lineal	—	(sin hiperparámetros)
Árbol de decisión	<code>max_depth</code>	3, 5, 10, 15
	<code>min_samples_leaf</code>	(defecto)
Bosque aleatorio	<code>n_estimators</code>	50, 100
	<code>max_depth</code>	10, 20
XGBoost	<code>n_estimators</code>	100, 300, 500
	<code>learning_rate</code>	0.01, 0.05, 0.10
	<code>max_depth</code>	3, 5

7.2 Resultados de los modelos de regresión

La Tabla 7.3 recoge las métricas obtenidas sobre el conjunto de validación, empleadas para comparar los modelos y seleccionar el ganador.

Cuadro 7.3: Resultados de los modelos de regresión sobre el conjunto de validación.

Modelo	R^2	MAE (€)	RMSE (€)
Regresión lineal	0,1438	63,33	84,04
XGBoost	0,5063	45,53	63,82
Árbol de decisión	0,5930	38,22	57,95
Bosque aleatorio	0,8670	19,42	33,12

El **bosque aleatorio** es el mejor modelo con amplia diferencia: $R^2 = 0,8670$ en validación, $MAE = 19,42 \text{ €}$, $RMSE = 33,12 \text{ €}$. El segundo mejor es el árbol de decisión ($R^2 = 0,59$), seguido de XGBoost ($R^2 = 0,51$) y de la regresión lineal ($R^2 = 0,14$). Evaluado sobre el conjunto de prueba independiente, el bosque aleatorio alcanza $R^2 = 0,8676$, $MAE = 19,43 \text{ €}$ y $RMSE = 33,05 \text{ €}$, sin indicios claros de sobreajuste entre validación y prueba.

Los resultados merecen varias observaciones. La regresión lineal obtiene un R^2 de apenas 0,14, confirmando que la relación entre las variables de entrada y el precio es fuertemente no lineal: un modelo que solo captura tendencias lineales solo explica aproximadamente el 14 % de la variabilidad observada en el precio. XGBoost, de forma inesperada, queda por debajo del árbol de decisión simple; esto podría deberse a que la rejilla explorada de hiperparámetros puede no haber encontrado la configuración óptima de *learning rate* y número de estimadores para este conjunto de datos, y a que el bosque aleatorio (cuya mayor flexibilidad en profundidad de árbol y número de estimadores fue más exhaustivamente ajustada) aprovecha mejor la estructura de los datos.

La regresión lineal, además de tener el peor R^2 ($R^2 = 0,14$), presenta el comportamiento esperado ante una relación no lineal entre tamaño de habitación y precio: la Figura 7.1 muestra el gráfico de dispersión de **tamaño_habitacion** (m^2) frente a **precio** (€) sobre el conjunto de entrenamiento, donde se aprecia claramente que la nube de puntos no sigue un patrón lineal y que la recta de regresión no capturaría la estructura de los datos.

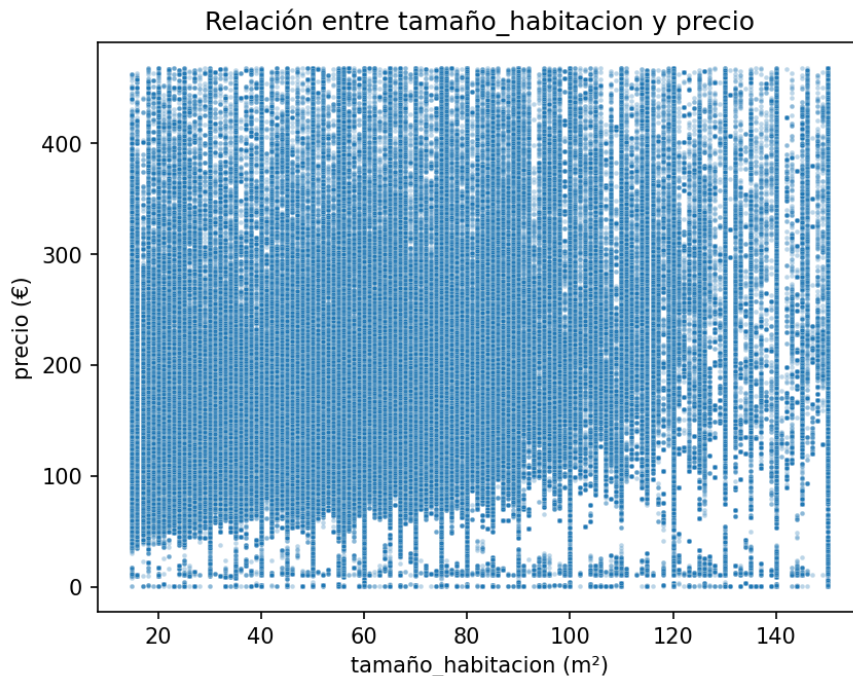


Figura 7.1: Diagrama de dispersión de **tamaño_habitacion** (m^2) frente a **precio** (€) sobre el conjunto de entrenamiento.

7.2.1 Variable raíz del árbol de decisión

La Figura 7.2 muestra los primeros cuatro niveles del árbol de decisión entrenado con los hiperparámetros óptimos sobre el conjunto de entrenamiento.

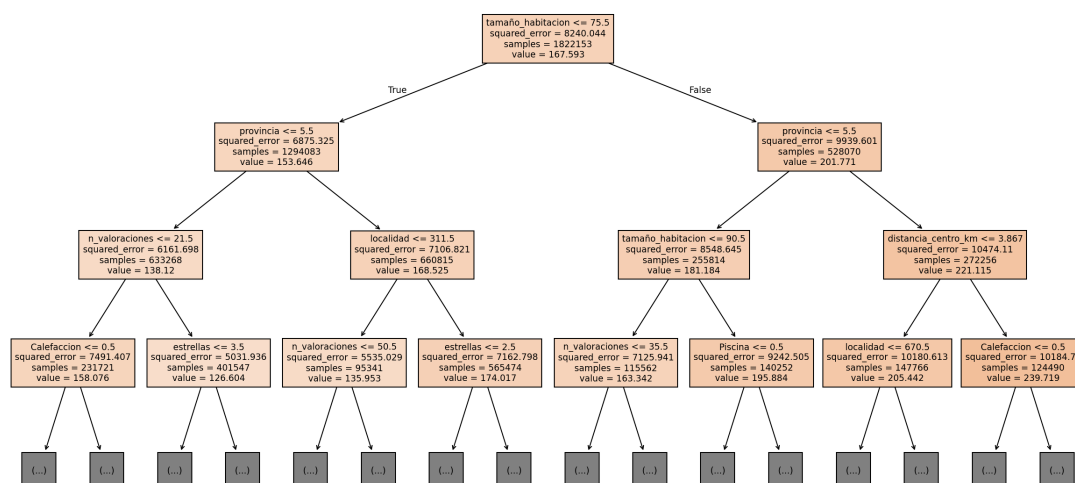


Figura 7.2: Árbol de decisión de regresión: primeros 4 niveles.

Nota: cada nodo muestra la variable de división, el error cuadrático en ese nodo, el número de muestras y el valor predicho (precio medio); los nodos naranjas son nodos internos y los grises son hojas. Cada división recorta el espacio de características en dos regiones más homogéneas en precio.

La raíz del árbol es la variable **tamaño_habitacion** con el umbral $\leq 75,5\text{m}^2$. Este resultado es consistente con que **tamaño_habitacion** sea la variable con mayor correlación lineal con el precio en la Figura 6.6, y confirma que es el predictor más potente del precio en este conjunto de datos, por encima incluso de la categoría por estrellas. La interpretación es intuitiva: alojamientos más grandes (suites, apartamentos amplios, casas con múltiples habitaciones) tienen precios sistemáticamente superiores independientemente de su clasificación oficial.

En el segundo nivel, la variable de división es **provincia** ($\leq 5,5$) en **ambas** ramas, lo que pone de manifiesto que la localización geográfica es el segundo factor más discriminante con independencia del tamaño del alojamiento. En el tercer nivel aparecen **n_valoraciones**, **localidad**, **tamaño_habitacion** (de nuevo, con umbral $\leq 90,5\text{m}^2$ para alojamientos grandes) y **distancia_centro_km**. En el cuarto nivel ya afloran variables de servicio como **Calefaccion** y **Piscina**, junto a **estrellas**, que capturan matices adicionales de la relación precio-atributo.

7.2.2 Importancia de las variables en el bosque aleatorio

El análisis de importancia de variables del bosque aleatorio según la reducción media de la impureza confirma la siguiente jerarquía de predictores:

1. **distancia_centro_km** y **tamaño_habitacion**: las dos variables más importantes, con importancias prácticamente idénticas (0,155 cada una). La paridad indica que la proximidad al centro urbano y el espacio del alojamiento son los factores determinantes del precio a partes iguales.
2. **n_valoraciones**: el número de reseñas es el tercer predictor más relevante (0,119), lo que refleja que los alojamientos con mayor historial de valoraciones tienen precios sistemáticamente distintos a los recién incorporados a la plataforma.
3. **localidad**: la ubicación a nivel de municipio captura diferencias de precio muy significativas entre destinos costeros y de interior (0,113).
4. **dias_restantes**: la antelación con la que se reserva el alojamiento tiene un efecto notable sobre el precio (0,091), capturando el comportamiento de precio dinámico de la plataforma.
5. **valoracion_clientes**: los alojamientos mejor valorados se asocian con precios más altos (0,084).

Los servicios binarios (Piscina, Restaurante, Gimnasio, etc.) tienen una importancia moderada individualmente, pero en conjunto contribuyen de forma sustancial a distinguir alojamientos de alto nivel de los de categoría media.

7.3 Distribución de categorías de chollo

Una vez obtenidas las predicciones del bosque aleatorio sobre el conjunto de prueba (390.462 registros), se calculó la etiqueta de chollo para cada observación aplicando la regla determinista descrita en la Sección 5.3. La distribución resultante se muestra en la Tabla 7.4.

Cuadro 7.4: Distribución de categorías de chollo en el conjunto de prueba (390.462 registros).

Categoría	Condición ($r = y/\hat{y}$)	Registros	Porcentaje
Hiper-chollo	$r \leq 0,75$	29.675	7,6 %
Super-chollo	$0,75 < r < 0,85$	35.923	9,2 %
Chollo	$0,85 \leq r < 0,97$	96.444	24,7 %
Normal	$0,97 \leq r \leq 1,03$	125.338	32,1 %
Inflado	$r > 1,03$	103.082	26,4 %
Total chollos (Chollo + Super + Hiper)		162.042	41,5 %

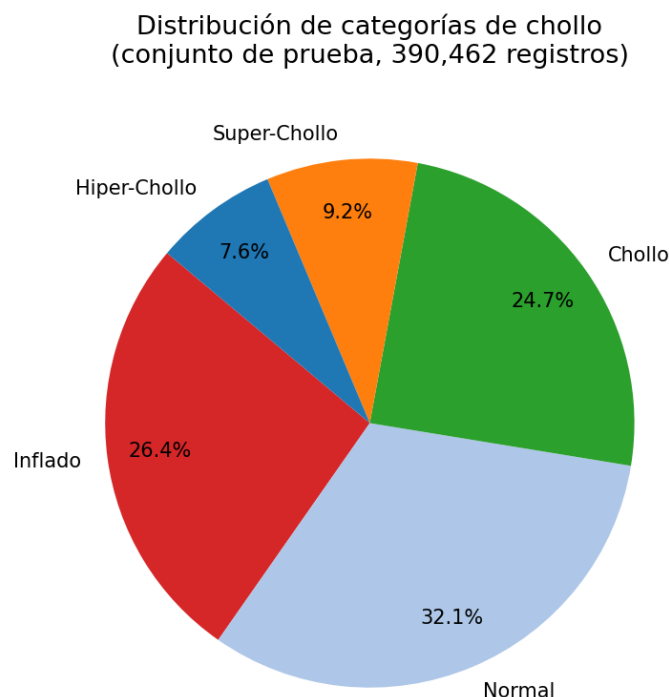


Figura 7.3: Distribución de las cinco categorías de chollo en el conjunto de prueba (390.462 registros).

Nota: valores predichos por el bosque aleatorio. Inflado 103.082 (26,4 %), Normal 125.338 (32,1 %), Chollo 96.444 (24,7 %), Super-Chollo 35.923 (9,2 %) e Hiper-Chollo 29.675 (7,6 %).

El resultado más llamativo es que **4 de cada 10 observaciones** (41,5 %) del conjunto de prueba se clasifica en alguna de las categorías de chollo ($r < 0,97$). La categoría más frecuente es «Normal» (32,1 %). Entre los chollos, el grueso cae en la categoría «Chollo» (precio entre un 3 % y un 15 % por debajo), mientras que las categorías «Super-chollo» e «Hiper-chollo» representan fracciones menores (9,2 % y 7,6 % respectivamente). El resultado refleja una variabilidad de precios real en la plataforma que el usuario individual difícilmente puede percibir sin el apoyo de un modelo predictivo.

7.4 Aplicación web

El sistema de detección de chollos se despliega como una aplicación web interactiva construida con Streamlit [30], accesible desde cualquier navegador sin necesidad de instalar software adicional.

7.4.1 Arquitectura de la aplicación

La aplicación se organiza en dos páginas principales:

- **Página principal (*Home*)**. Presenta un mapa coroplético interactivo de Andalucía construido con Plotly [25], que muestra el número de alojamientos disponibles por provincia. El mapa combina un coroplético por provincias (GeoJSON con los límites administrativos) con una capa de puntos dispersos correspondientes a la ubicación GPS de cada alojamiento único del *dataset* histórico. El fichero HTML resultante se genera durante la fase de visualización y se reutiliza como mapa por defecto en la aplicación, evitando recalcularlo en cada sesión.
- **Página de búsqueda (*Búsqueda*)**. Página principal de interacción donde el usuario especifica sus preferencias y obtiene los resultados categorizados por grado de oportunidad.

7.4.2 Flujo de trabajo de la página de búsqueda

El flujo completo desde que el usuario introduce sus preferencias hasta que visualiza los resultados consta de seis pasos:

1. **Formulario de preferencias**: el usuario selecciona provincia o localidad de destino, rango de fechas de la estancia, tipo de alojamiento y hasta 16 filtros de servicios (piscina, spa, mascotas, desayuno incluido, etc.) mediante controles interactivos de Streamlit. El formulario incluye además un control deslizante de tres posiciones para elegir el nivel de exigencia de la categorización de chollo (Tabla 7.5), acompañado de un botón de información que detalla el significado de cada nivel.
2. **Extracción en tiempo real**: al pulsar el botón de búsqueda se lanza un módulo de recopilación de datos asíncrono, que obtiene hasta 25 alojamientos por localidad seleccionada.
3. **Preprocesamiento**: los datos recién extraídos pasan por la misma secuencia de transformaciones aplicada durante el entrenamiento: extracción de servicios binarios, cálculo de la distancia Haversine al centro urbano e indicadores temporales.
4. **Codificación**: se aplican los codificadores serializados con `joblib` durante el entrenamiento (`le_localidad.pkl`, `le_provincia.pkl`, `columnas_modelo.pkl`), garantizando que el espacio de características de la predicción sea idéntico al del entrenamiento.
5. **Predicción y categorización**: una versión podada del bosque aleatorio (`bosque_aleatorio_reg_ligero.pkl`, 20 de los 100 árboles originales del modelo entrenado; véase la Sección 7.4.3) predice el precio por noche justo para cada alojamiento. A continuación se calcula el ratio $r = y/\hat{y}$ y se asigna

la etiqueta de chollo según los umbrales del nivel de exigencia elegido por el usuario en el paso 1 (Tabla 7.5). Al pesar unos 59 MB comprimido, muy por debajo del límite de 100 MB de GitHub, el modelo podado se aloja directamente en el repositorio del proyecto y se carga desde disco al arrancar la aplicación, sin depender de una descarga remota.

6. **Presentación de resultados:** los alojamientos se presentan en una tabla interactiva ordenada por grado de ahorro, con columnas para el nombre del alojamiento, enlace a la ficha, precio real, precio predicho y categoría. Un gráfico de tarta con `plotly.express` muestra la distribución de categorías entre los resultados encontrados.

Los umbrales de categorización descritos en la Sección 5.3 no son fijos en la aplicación: el usuario puede endurecerlos mediante el control deslizante mencionado en el paso 1, exigiendo un mayor porcentaje de ahorro para que un alojamiento se etiquete como chollo. La Tabla 7.5 recoge el rango de ahorro que exige cada categoría en los tres niveles disponibles. Cambiar de nivel no requiere repetir la extracción de datos ni la predicción: al operar sobre el precio real y el precio predicho ya calculados, la recategorización es instantánea.

Cuadro 7.5: Rango de ahorro exigido por categoría según el nivel de exigencia elegido por el usuario.

Categoría	Predeterminado	Exigente	Muy exigente
Hiper-chollo	más de 25 %	más de 35 %	más de 50 %
Super-chollo	15 %–25 %	20 %–35 %	30 %–50 %
Chollo	3 %–15 %	10 %–20 %	15 %–30 %
Normal	precio justo (± 3 %)	ahorro < 10 %	ahorro < 15 %

Nota: la categoría Inflado no tiene ahorro; el precio real es al menos un 3 % superior al estimado (igual en los tres niveles).

7.4.3 Optimización de memoria: poda del modelo para el despliegue

El bosque aleatorio ganador (100 árboles, Sección 4.4) ocupa aproximadamente 1,3 GB en disco, muy por encima del límite de 100 MB por archivo que impone GitHub, lo que impedía alojarlo directamente en el repositorio del proyecto. La solución inicial consistió en alojar el modelo en un repositorio público de Hugging Face Hub [15], descargándolo en tiempo de ejecución la primera vez que arrancaba la aplicación; esto resolvía el problema de tamaño de archivo, pero no anticipaba un problema distinto que se manifestó una vez desplegada la aplicación: el modelo ocupa cerca de 1,4 GB en memoria RAM una vez deserializado con `joblib`, muy por encima del límite (aproximadamente 1 GB) del plan gratuito de Streamlit Community Cloud empleado para el despliegue, lo que provocaba el cierre abrupto de la aplicación al cargar el modelo, incluso antes de que el usuario realizara ninguna búsqueda. El origen del modelo (disco local o descarga remota desde

Hugging Face Hub) resultó irrelevante para este segundo problema, ya que el pico de memoria se producía al cargar el modelo en RAM, no al descargarlo.

La solución finalmente adoptada explota una propiedad estructural del bosque aleatorio: la predicción es el promedio de las predicciones de los B árboles individuales que lo componen (Sección 4.4), por lo que es posible reducir el número de árboles del *ensemble después* del entrenamiento, sin necesidad de reentrenar el modelo, simplemente descartando una parte de los árboles ya ajustados. Se conservaron los primeros 20 de los 100 árboles entrenados, reduciendo la huella de memoria a aproximadamente 373 MB (una cuarta parte del original), cifra que sí resulta compatible con el presupuesto de memoria disponible.

La Tabla 7.6 compara el rendimiento del modelo completo y de la versión podada sobre el conjunto de prueba. La pérdida de rendimiento es mínima: el R^2 desciende de 0,8676 a 0,8634 y el MAE aumenta en apenas 0,36 €. Esta pequeña pérdida es plenamente asumible, dado que el modelo podado sigue superando con amplitud al segundo mejor modelo evaluado en este trabajo, el árbol de decisión ($R^2 = 0,5930$, Tabla 7.3): reducir el número de árboles del bosque aleatorio a una quinta parte resulta, por tanto, preferible a sustituirlo por un modelo con un rendimiento sensiblemente inferior.

Cuadro 7.6: Comparación del bosque aleatorio completo y de la versión podada (20 árboles) sobre el conjunto de prueba.

Versión	Árboles	R^2	MAE (€)	RMSE (€)
Completo	100	0,8676	19,43	33,05
Podado	20	0,8634	19,79	33,57

Además de resolver el problema de memoria, la poda aporta una ventaja adicional para el despliegue: el archivo serializado se reduce de 1,3 GB a aproximadamente 59 MB comprimido, muy por debajo del límite de 100 MB de GitHub. Esto permite alojar el modelo directamente en el repositorio del proyecto, eliminando la dependencia de Hugging Face Hub y la descarga en tiempo de ejecución de la solución anterior: el modelo se carga desde disco al arrancar la aplicación, sin necesitar red ni gestionar los límites de tasa de peticiones no autenticadas propios de un servicio externo.

El bosque aleatorio completo (100 árboles) se mantiene como el modelo de referencia de este trabajo, y es el que se reporta en la Tabla 7.3; la versión podada se emplea exclusivamente en el despliegue de la aplicación web, como solución de compromiso frente a las restricciones de memoria del hosting gratuito.

7.4.4 Despliegue

La aplicación se despliega en Streamlit Community Cloud, vinculada al repositorio Git del proyecto. Las dependencias de Python se gestionan con `uv` y se especifican en `pyproject.toml`; las dependencias del sistema operativo (necesarias para Playwright) se incluyen en `packages.txt`. El despliegue es continuo: cualquier cambio en la rama principal desencadena automáticamente un redepliegue de la aplicación.

7.4.5 Limitaciones de la extracción en tiempo real

Los servicios de habitación (`Aire`, `Calefaccion`, `Vistas`, `Terraza`, `Baño_privado`) se extraen de la oferta más barata disponible en el instante concreto de la consulta. Cada búsqueda del usuario abre una sesión HTTP nueva y anónima frente a la plataforma, sin cookies persistentes entre una ejecución y la siguiente, por lo que la plataforma puede aplicar experimentación o personalización a nivel de sesión (orden de las ofertas, promociones visibles) que resulte en que la oferta identificada como «más barata» (y, por tanto, sus servicios asociados) no sea siempre la misma habitación física de una búsqueda a otra, aunque el intervalo entre ambas sea de pocos minutos. En consecuencia, estas cinco variables binarias pueden variar ligeramente entre ejecuciones repetidas de la misma búsqueda, lo que se traslada a pequeñas diferencias en el precio predicho y en la categoría de chollo asignada. Los servicios generales del alojamiento (`Piscina`, `Parking`, `Restaurante`, etc.), al pertenecer al establecimiento completo y no a una habitación concreta, no se ven afectados por esta limitación.

7.5 Conclusiones

Este trabajo ha presentado el diseño, implementación y evaluación de un sistema integral para la detección de alojamientos turísticos con precio real inferior al valor predicho en Andalucía. Las principales conclusiones son las siguientes:

1. **El precio de un alojamiento no es aleatorio.** El bosque aleatorio explica el 86,8 % de la varianza del precio ($R^2 = 0,8676$) con un MAE de 19,43 €por noche, lo que demuestra que el precio depende de forma predecible del tamaño de la habitación, la ubicación y la temporalidad. Esta predictibilidad es la condición necesaria para que el sistema de detección de chollos sea útil.
2. **4 de cada 10 alojamientos se clasifican como chollo.** El 41,5 % de los alojamientos del conjunto de prueba se clasifica en alguna de las categorías de chollo ($r < 0,97$). Esto indica que existe una variabilidad de precios real en la plataforma que el usuario ordinario no puede percibir sin apoyo de aprendizaje automático.

3. **La calidad del dato es determinante para el rendimiento del sistema.** La recopilación de datos en dos fases, la geocodificación, la extracción de 12 servicios binarios, la gestión rigurosa de outliers y los indicadores temporales fueron más determinantes para el resultado final que la elección del algoritmo de regresión. La consistencia entre el R^2 en validación (0,8670) y en prueba (0,8676), con una diferencia inferior a 0,001, no muestra indicios claros de sobreajuste entre validación y prueba, mientras que el preprocesamiento descuidado podría haber elevado el error en varios euros.
4. **La arquitectura *end-to-end* es el verdadero reto de ingeniería.** Conectar la recopilación de datos, el preprocesamiento, la modelización y la aplicación web en un único sistema reproducible (donde los encoders, escaladores y el modelo comparten exactamente el mismo espacio de características) es más complejo que cualquiera de sus componentes por separado. El diseño modular adoptado (un módulo por responsabilidad, serialización con `joblib`, gestión de entorno con `uv`) fue clave para lograrlo.

El código completo de este trabajo está disponible públicamente en el repositorio <https://github.com/Marjimleo3/Trabajo-Fin-de-Grado> [16].

7.6 Trabajo futuro

Las siguientes líneas de trabajo podrían extender y mejorar el sistema presentado.

1. **Ampliación a otras plataformas y territorios.** La arquitectura modular del sistema facilitaría extender la cobertura a otras comunidades autónomas o incorporar otras plataformas de alojamiento, lo que enriquecería el conjunto de datos y permitiría comparaciones entre plataformas.
2. **Actualización periódica del modelo.** Dado que los precios en las OTA son altamente dinámicos, un sistema en producción requeriría reentrenamiento periódico con datos recientes. La integración de un sistema de orquestación de flujos de trabajo permitiría automatizar este proceso.
3. **Incorporación de variables de texto.** Las reseñas de los usuarios y las descripciones de los alojamientos contienen información semántica no capturada por las variables estructuradas actuales. Modelos de lenguaje preentrenados podrían extraer *embeddings* de texto que mejoren la predicción del precio.
4. **Alertas automáticas de chollo.** Una extensión natural de la aplicación sería un sistema de alertas que notifique al usuario cuando aparezca un alojamiento categorizado como «Hiper-chollo» que se ajuste a sus preferencias almacenadas, sin necesidad de que el usuario inicie la búsqueda manualmente.
5. **Aprendizaje automático no supervisado.** La detección de anomalías mediante métodos no supervisados (*Isolation Forest*, *Local Outlier Factor*)

sobre los precios de la plataforma podría complementar el enfoque supervisado actual, especialmente para detectar chollos en categorías de alojamiento con pocos datos históricos.

Bibliografía

- [1] ABRATE, G., CAPRIELLO, A., AND FRAQUELLI, G. When quality signals talk: Evidence from the turin hotel industry. *Tourism Management* 32, 4 (2011), 912–921. <https://doi.org/10.1016/j.tourman.2010.08.006>.
- [2] AGGARWAL, C. C. *Outlier Analysis*, 2nd ed. Springer, New York, 2017. <https://doi.org/10.1007/978-3-319-47578-3>.
- [3] BREIMAN, L. Random forests. *Machine Learning* 45, 1 (2001), 5–32. <https://doi.org/10.1023/A:1010933404324>.
- [4] BREIMAN, L., FRIEDMAN, J. H., OLSHEN, R. A., AND STONE, C. J. *Classification and Regression Trees*. Chapman and Hall/CRC, New York, 1984.
- [5] BREUNIG, M. M., KRIEGEL, H.-P., NG, R. T., AND SANDER, J. LOF: Identifying density-based local outliers. *Proceedings of the ACM SIGMOD International Conference on Management of Data* (2000), 93–104. <https://doi.org/10.1145/335191.335388>.
- [6] BUHALIS, D., AND LAW, R. Progress in information technology and tourism management: 20 years on and 10 years after the internet—the state of etourism research. *Tourism Management* 29, 4 (2008), 609–623.
- [7] CASELLA, G., AND BERGER, R. L. *Statistical Inference*, 2nd ed. Duxbury Press, Pacific Grove, California, 2002.
- [8] CHANDOLA, V., BANERJEE, A., AND KUMAR, V. Anomaly detection: A survey. *ACM Computing Surveys* 41, 3 (2009), 1–58. <https://doi.org/10.1145/1541880.1541882>.
- [9] CHEN, T., AND GUESTRIN, C. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (New York, 2016), ACM, pp. 785–794. <https://doi.org/10.1145/2939672.2939785>.
- [10] ENZ, C. A., CANINA, L., AND LOMANNO, M. Competitive pricing decisions in uncertain times. *Cornell Hospitality Quarterly* 50, 3 (2009), 325–341. <https://doi.org/10.1177/1938965509338550>.

- [11] FRIEDMAN, J. H. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics* 29, 5 (2001), 1189–1232. <https://doi.org/10.1214/aos/1013203451>.
- [12] HASTIE, T., TIBSHIRANI, R., AND FRIEDMAN, J. H. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2nd ed. Springer Series in Statistics. Springer, New York, 2009.
- [13] HINTON, G. E., AND SALAKHUTDINOV, R. R. Reducing the dimensionality of data with neural networks. *Science* 313, 5786 (2006), 504–507. <https://doi.org/10.1126/science.1127647>.
- [14] HU, T., AND SONG, H. Analysis of influencing factors and distribution simulation of budget hotel room pricing based on big data and machine learning from a spatial perspective. *Sustainability* 15, 1 (2023), 617. <https://doi.org/10.3390/su15010617>.
- [15] HUGGING FACE, INC. Hugging face hub. <https://huggingface.co/docs/hub>, 2026. Accedido en julio de 2026.
- [16] JIMÉNEZ LEÓN, M. Detector de chollos: Trabajo de fin de grado. <https://github.com/Marjimleo3/Trabajo-Fin-de-Grado>, 2026. Accedido en julio de 2026.
- [17] LIU, F. T., TING, K. M., AND ZHOU, Z.-H. Isolation forest. In *Proceedings of the 8th IEEE International Conference on Data Mining* (2008), IEEE, pp. 413–422. <https://doi.org/10.1109/ICDM.2008.17>.
- [18] LLANEZA HESSE, C., AND RAYA VÍLCHEZ, J. M. The effect of COVID-19 on the peer-to-peer rental market. *Tourism Economics* 28, 1 (2022), 222–247. <https://doi.org/10.1177/13548166211044229>.
- [19] MICROSOFT CORPORATION. Playwright for python. <https://playwright.dev/python>, 2023. Accedido en julio de 2026.
- [20] MONTGOMERY, D. C., PECK, E. A., AND VINING, G. G. *Introduction to Linear Regression Analysis*, 5th ed. Wiley, Hoboken, New Jersey, 2012.
- [21] OLSTON, C., AND NAJORK, M. Web crawling. *Foundations and Trends in Information Retrieval* 4, 3 (2010), 175–246. <https://doi.org/10.1561/15000000017>.
- [22] OPENSTREETMAP FOUNDATION. Nominatim geocoding service. <https://nominatim.org>, 2023. Accedido en julio de 2026.
- [23] PEARSON, K. Notes on regression and inheritance in the case of two parents. *Proceedings of the Royal Society of London* 58 (1895), 240–242.

-
- [24] PEDREGOSA, F., VAROQUAUX, G., GRAMFORT, A., MICHEL, V., THIRION, B., GRISEL, O., BLONDEL, M., PRETTENHOFER, P., WEISS, R., DUBOURG, V., VANDERPLAS, J., PASSOS, A., COURNAPEAU, D., BRUCHER, M., PERROT, M., AND DUCHESNAY, É. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.
- [25] PLOTLY TECHNOLOGIES INC. Collaborative data science. <https://plotly.com>, 2023. Accedido en julio de 2026.
- [26] REZAZADEH KALEHBASTI, P., NIKOLENKO, L., AND REZAEI, H. Airbnb price prediction using machine learning and sentiment analysis. In *Machine Learning and Knowledge Extraction (CD-MAKE 2021)* (Cham, 2021), vol. 12844 of *Lecture Notes in Computer Science*, Springer. https://doi.org/10.1007/978-3-030-84060-0_11.
- [27] SÁNCHEZ-LOZANO, G., PEREIRA, L. N., AND CHÁVEZ-MIRANDA, E. Big data hedonic pricing: Econometric insights into room rates’ determinants by hotel category. *Tourism Management* 85 (2021), 104308. <https://doi.org/10.1016/j.tourman.2021.104308>.
- [28] SELENIUM PROJECT. Selenium browser automation project. <https://www.selenium.dev>, 2023. Accedido en julio de 2026.
- [29] SOLER-GARCÍA, I. P., AND GEMAR-CASTILLO, G. Hedonic price models with geographically weighted regression: An application to hospitality. *Journal of Destination Marketing & Management* 9 (2018), 126–137. <https://doi.org/10.1016/j.jdmm.2017.12.001>.
- [30] STREAMLIT INC. Streamlit: The fastest way to build and share data apps. <https://streamlit.io>, 2023. Accedido en julio de 2026.
- [31] TALLURI, K. T., AND VAN RYZIN, G. J. *The Theory and Practice of Revenue Management*. Springer, New York, 2004.
- [32] TUKEY, J. W. *Exploratory Data Analysis*. Addison-Wesley, Reading, Massachusetts, 1977.
- [33] VALDIVIA, A., LUZÓN, M. V., AND HERRERA, F. Sentiment analysis in TripAdvisor. *IEEE Intelligent Systems* 32, 4 (2017), 72–77. <https://ieeexplore.ieee.org/iel7/9670/8012276/08012330.pdf>.
- [34] WEI, T., AND SIMKO, V. R package `corrplot`: Visualization of a correlation matrix. <https://github.com/taiyun/corrplot>, 2021. Accedido en julio de 2026.